Proyecto Final UD 1

Alberto Piedra Venegas

8 de noviembre de 2025

Índice

1. Introducción **4**
- 1.1. Contextualización . . . 4
- 1.2. Justificación . . . 4
- 1.3. Objetivos . . . 4
- 1.4. Alcance del proyecto . . . 5
- 1.5. Planificación . . . 5

2. Marco teórico **6**
- 2.1. Apache Kafka . . . 6
- 2.2. Apache NiFi . . . 6
- 2.3. MongoDB . . . 7
- 2.4. Dremio . . . 8
- 2.5. Flask . . . 8

3. Fase de análisis **9**

4. Fase de diseño **10**
- 4.1. Diseño de la Persistencia en MongoDB . . . 10
- 4.2. Diseño del flujo de datos en NiFi . . . 11
- 4.3. Diseño de la aplicación Web Flask . . . 12

5. Fase de implementación **13**
- 5.1. Configuración del entorno . . . 13
  - 5.1.1. Instalación Apache Kafka . . . 17
  - 5.1.2. Instalación MongoDB . . . 20
  - 5.1.3. Instalación de Flask . . . 22
  - 5.1.4. Instalación de Dremio . . . 22
- 5.2. Generación del Flujo de Trabajo con Nifi . . . 24
  - 5.2.1. Fase de ingesta de Datos . . . 24
  - 5.2.2. Fase de manejo de datos . . . 25
  - 5.2.3. Fase de interfaz web . . . 31
  - 5.2.4. Fase de analisis . . . 32

6. Fase de pruebas **39**

7. Conclusiones y líneas futuras **44**
- 7.1. Conclusiones . . . 44
- 7.2. Lineas futúras . . . 44
  - 7.2.1. Evolución de la ingesta de datos . . . 45
  - 7.2.2. Optimización de la interfaz web . . . 45
  - 7.2.3. Machine learning y analisis predictivo . . . 45

8. Bibliografía **46**

9. Diario de trabajo **46**

10.Anexos varios **47**

1. Introducción

1.1. Contextualización

Como nuevo miembro del equipo de SmartCity Cádiz, nuestra tarea esencial consiste en el diseño, desarrollo e implementación de SmartParkingFlow, un sistema avanzado y distribuido, enmarcado dentro de la estrategia de Internet de las Cosas (IoT), para la recolección, almacenamiento y visualización de datos generados por una red de sensores de un aparcamiento inteligente.

El concepto de Smart City se fundamenta en la capacidad de procesar grandes volúmenes de datos urbanos en tiempo real para optimizar los servicios y mejorar la calidad de vida. En este contexto, SmartParkingFlow se convierte en la infraestructura de datos clave para la gestión de uno de los recursos más críticos de la ciudad: el espacio de estacionamiento. El sistema propuesto debe demostrar robustez, alta disponibilidad y escalabilidad para manejar el flujo continuo de información proveniente de múltiples dispositivos IoT. Utilizaremos tecnologías de vanguardia en el procesamiento de streaming de datos para garantizar que la información refleje fielmente la realidad operativa del aparcamiento.

1.2. Justificación

El proyecto «SmartParkingFlow» nace de la necesidad de mejorar el estacionamiento en la ciudad de Cádiz. Además, este proyecto permitiría disponer de información precisa y en tiempo real de los distintos aparcamientos, lo cual facilitaría la vida no solo a los ciudadanos, sino también a los empleados de la administración pública. Actualmente, la falta de información en tiempo real sobre la disponibilidad de plazas provoca varios problemas urbanísticos y ambientales significativos:

- **Congestión del trafico** Se estima que una gran porción de la congestión en el trafico es debido a los conductores reduciendo la velocidad para buscar aparcamientos.
- **Ineficiencia administrativa** La administración publica carece de metricas historicas precisas sobre los patrones de uso, esto dificulta la planificación de nuevos aparcamientos y la optimización de los servicios de mantenimiento.

1.3. Objetivos

Este proyecto tiene dos objetivos principales:

1. La utilización de **Apache Kafka** para gestionar la recepción de mensajes continuos enviados por los sensores instalados en cada plaza del aparcamiento. Estos sensores enviarán datos sobre si la plaza está ocupada o no, y otros datos de utilidad, como: temperatura, nivel del aparcamiento o la batería del dispositivo.
2. La creación de un sistema que **almacene y gestione los datos**, tanto del estado de los aparcamientos en el momento, como un histórico del estado de los aparcamientos a lo largo del tiempo.

1.4. Alcance del proyecto

Al final de este proyecto, se espera tener un flujo de datos utilizando **Apache NiFi** que automatizará el proceso de ingestión de los datos en Kafka hasta la inserción de los datos en distintas colecciones de **MongoDB**. Además, también se creará una página web utilizando **Flask** para tener un mapa visual del estado de los aparcamientos en tiempo real.

1.5. Planificación

El primer paso para la realización de este proyecto será la instalación de todos los programas que se utilizarán: Apache NiFi, Apache Kafka, MongoDB, Flask y por último Dremio.

Luego de tener todos los programas instalados, se procederá a crear el flujo dentro de Apache NiFi. Este flujo consistirá en la generación de documentos con información provista por los sensores de los aparcamientos. Estos documentos se enviarán a través de NiFi a dos colecciones distintas dentro de MongoDB:

1. La primera colección reflejará el **estado actual** y nos dirá si las plazas están libres u ocupadas.

2. La otra colección será un **histórico** con los datos registrados por los sensores.

Una vez que el flujo esté hecho, el siguiente paso será la creación de una aplicación web con Flask que consuma los datos de la colección de MongoDB para mostrar un mapa visual del estado de cada plaza del aparcamiento. Por último, se configurará Dremio para poder realizar consultas descriptivas sobre la ocupación del parking.

2. Marco teórico

En este apartado, estableceremos los fundamentos teóricos y las justificaciones de las tecnologías que hemos utilizado para cada apartado del proyecto. La elección de estas herramientas se basa en la necesidad de procesar los datos de los sensores en tiempo real con alta disponibilidad y escalabilidad.

2.1. Apache Kafka

Apache Kafka se trata de una plataforma distribuida para la transmisión de datos que permite no solo publicar, almacenar y procesar flujos de eventos de forma inmediata, sino también suscribirse a ellos. Está diseñada para administrar los flujos de datos de varias fuentes y enviarlos a distintos usuarios. Esta plataforma es muy útil para nuestro proyecto ya que nos permite, no solo manejar los datos que nos lleguen desde los distintos sensores sino también poder almacenarlos en distintas bases de datos.

Conceptos clave de Kafka

- **Productor (Producer)**: Es el dispositivo encargado de enviar los datos a un *topic*. En nuestro caso, será el encargado de enviar los datos que recibimos desde los sensores a un *topic* que nosotros creemos.
- **Consumidor (Consumer)**: Es el encargado de leer los datos que llegan a los *topics*. En nuestro proyecto se encarga de guardar los datos para luego poder enviarlos a las distintas bases de datos.
- **Topic**: Es el lugar donde el productor publica los mensajes para que el consumidor pueda leerlos en otro momento.

2.2. Apache NiFi

Apache NiFi se trata de una herramienta de software que sirve para automatizar el flujo de datos entre distintos sistemas de software. En nuestro proyecto, utilizaremos Apache NiFi para crear un flujo de datos que, mediante el uso de sus propios procesadores, simule los datos de los sensores de los aparcamientos y los lleve hasta las distintas bases de datos que utilizaremos para guardar estos datos y luego poder usarlos tanto en la generación de la interfaz web como para análisis posteriores.

Conceptos clave de NiFi

- **FlowFile**: Son los paquetes de datos que fluyen a través del sistema y contienen los datos que usaremos.
- **Procesadores (Processors)**: Son componentes que realizan el trabajo del procesamiento de datos. Más tarde se explicarán los distintos procesos que hemos utilizado para este proyecto.
- **Conexiones**: Conectan los distintos procesadores y gestionan la presión de flujo para evitar que el sistema se sature.

La razón de usar NiFi es que facilita mucho la división de flujo de datos para los dos destinos necesarios en nuestro proyecto (base de datos con histórico y base de datos con estados actuales), además de que se puede ver visualmente todo el proceso y cómo los datos pasan entre los distintos procesos, lo que ayuda a identificar errores.

Procesadores Usados

1. **GenerateFlowFile**: Genera un archivo de texto vacío cada 5 segundos, que será enviado al siguiente proceso.

2. **ExecuteScript**: Ejecuta un script (en lenguaje Groovy) para simular los datos que nos llegarían desde los sensores de cada parking. Este código se encontrará en el apartado de Anexos Varios.

3. **PublishKafka**: Recibe los datos en formato JSON y los envía a un *topic* llamado `AparcamientosSmartCity`.

4. **ConsumeKafka**: Se encarga de leer los datos que han sido publicados en el *topic* anterior.

5. **PutMongo**: Es el último procesador de nuestro flujo. Se encarga de añadir los datos que recibe a las bases de datos que le indiquemos. Utilizaremos dos procesadores PutMongo: uno para la base de datos actual (con `upserts`) y otro para el histórico.

2.3. MongoDB

MongoDB es una base de datos **NoSQL** que almacena datos en documentos flexibles. Utilizaremos esta base de datos debido a la flexibilidad que nos ofrece, ya que no requiere de un esquema fijo como sí lo hacen las bases de datos relacionales. Esto es muy beneficioso para nuestro proyecto, ya que en un futuro los datos que nos lleguen desde los sensores podrían añadir o cambiar los campos de datos y, gracias a MongoDB, no necesitaríamos realizar migraciones complejas.

Patrón de Actualización Upsert

Para la base de datos que refleja los estados del parking a casi tiempo real utilizaremos el patrón de actualización `upsert`, el cual es una combinación de «Update» e «Insert».

El patrón funciona de la siguiente manera: El patrón recibirá los datos de un sensor del aparcamiento, buscará la clave primaria (en nuestro caso, el `bay_id`) y comprobará en la base de datos actual si ese `bay_id` ya existe. Si existe, actualizará los datos del documento con los nuevos que le han llegado; si no existe, lo añadirá a la base de datos. Este patrón garantiza que la base de datos que guarda los estados actuales del parking no tenga duplicados para la misma plaza y que siempre tenga la información más reciente del estado.

2.4. Dremio

Dremio es una plataforma de análisis de datos *lakehouse* de código abierto que permite a los usuarios consultar datos directamente desde el almacenamiento sin necesidad de moverlos y transformarlos previamente. Para el proyecto, se utilizará Dremio para realizar distintas consultas a las bases de datos y con ellas realizar análisis que permitan mejorar el funcionamiento del proyecto.

2.5. Flask

Flask es un *framework* para el desarrollo de proyectos web en Python, cuenta con un enfoque minimalista y está orientado para un tipo de trabajos más específicos como servicios web y microservicios.

Para nuestro proyecto, se diseñará una interfaz REST que realizará consultas cada 10 segundos a la colección dentro de la base de datos que guarda los estados de cada plaza del parking. Estos datos se utilizarán mediante un HTML estilado para facilitar que las personas puedan revisar las distintas plazas de aparcamiento. Para la actualización a casi tiempo real se utilizará la técnica de **Polling**.

3. Fase de análisis

Para realizar este proyecto primero debemos analizar el problema que queremos resolver, qué necesidades tendrá el proyecto y cómo deben interactuar cada parte que utilicemos.

En nuestro caso, el problema es la falta de información en tiempo real del estado de los distintos aparcamientos que hay en la ciudad de Cádiz, lo que provoca que la gente tenga problemas a la hora de buscar aparcamiento, ralentizando el tráfico. Con este proyecto intentaremos mejorar el flujo del tráfico, otorgando una página web que se vaya actualizando periódicamente con los estados de todos los aparcamientos que tengamos en Cádiz para que la gente pueda revisar los lugares con aparcamientos libres, reduciendo así parcialmente los estancamientos en el tráfico.

Para realizar este proyecto, lo primero que necesitaremos serán unos sensores que puedan capturar el estado de los aparcamientos, así como otros datos de ayuda. Estos datos, luego serán procesados para evitar problemas, y finalmente guardados en dos bases de datos distintas:

- Una con los **estados actuales** de cada aparcamiento, que será la que utilicemos para mostrar en nuestra página web.
- Otra que guardará **todos los cambios** en los estados de los aparcamientos para poder utilizarse en un futuro para realizar análisis y conclusiones que permitan mejorar el funcionamiento de la aplicación y el propio parking.

Para asegurarnos de que el proyecto funciona correctamente, debemos asegurarnos de que se cumplan ciertas características:

- Que los datos fluyan en **tiempo real**, para que la visualización en la página web sea lo más fiel a la realidad.
- Que se **procesen correctamente los datos** que nos llegan desde los sensores, para evitar datos erróneos o mensajes duplicados.
- Que la web sea **simple e intuitiva** para que los ciudadanos no tengan problemas al utilizarla.

4. Fase de diseño

Para el diseño de este proyecto, podremos dividirlo en cuatro fases:

Fase de Ingesta de Datos

La ingesta se simulará en **Apache NiFi** mediante un procesador que generará documentos **JSON** con características similares a las que darían los sensores. Estos datos se generarán aleatoriamente utilizando un código creado en el lenguaje de programación Groovy. En esta misma fase, se realizarán los procesamientos necesarios para que estos datos no tengan valores erróneos o estén duplicados.

El JSON que crearemos tendra estos campos que nos otorgaran la información que necesitaremos

Fase de Manejo de Datos

Se seguirá utilizando **Apache NiFi** para generar un flujo que, mediante distintos procesos, maneje los datos desde su generación hasta su ingreso en **Apache Kafka** y, posteriormente, en **MongoDB**. Se creará una copia de cada estado del aparcamiento: una se guardará en la colección de estado actual (con `upsert`) y la otra en la colección histórica.

Fase de Interfaz Web

Se creará una interfaz web con **Flask** para que los ciudadanos puedan acceder a ver los estados de los aparcamientos. La aplicación web utilizará la librería `PyMongo` para conectarse a la base de datos. La página web mostrará los datos mediante tarjetas, donde cada una representará una plaza y su estado (ocupada o libre) mediante el color de su contorno.

Fase de Análisis

Se utilizará la herramienta de **Dremio** para realizar consultas a los datos que están en la base de datos histórica y analizar la información para llegar a distintas conclusiones.

4.1. Diseño de la Persistencia en MongoDB

Se diseñarán dos colecciones dentro de MongoDB para responder al requisito de persistencia histórica y mantenimiento del estado actual.

Colección 1: `Parking_historico_db`

Esta colección se encargará de almacenar cada mensaje que nos llegue desde los sensores y será usado en el futuro para realizar análisis a largo plazo con Dremio.

- **Clave Primaria**: Generada automáticamente por MongoDB (`_id`).
- **Índice Compuesto**: Indexación de los campos `parking_id`, `bay_id`, `last_event_ts` para optimizar las consultas con Dremio.

Colección 2: `Parking_actual_db`

Esta colección se encargará de mantener solo el estado más reciente de cada plaza.

- **Clave de Búsqueda para Upsert**: Se definirá un índice único compuesto de los campos `parking_id` y `bay_id`. Esto garantizará que no haya duplicaciones, sino que se actualice si ya existe el documento y, si no existe, se cree.
- **Campos Conservados**: Se conservarán otros campos claves como: `occupied`, `level` y las otras métricas para poder realizar consultas con Dremio más fácilmente.

4.2. Diseño del flujo de datos en NiFi

El flujo de NiFi es el componente más importante del proyecto, ya que se encarga de la ingesta de datos, el movimiento y el guardado de los mismos en las bases de datos.

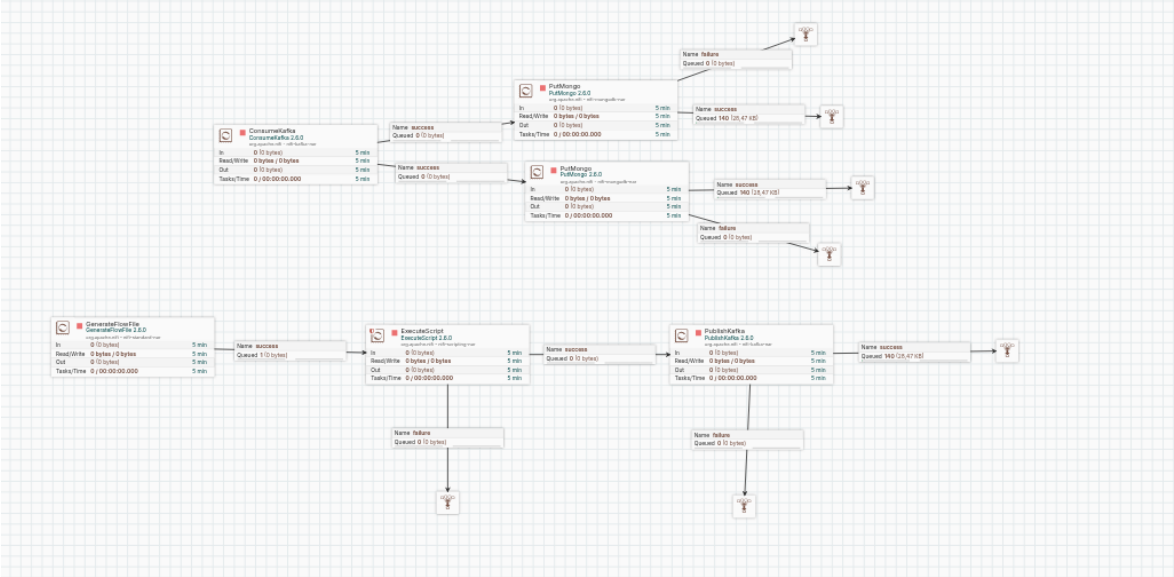


Figura 1: Flujo Nifi

El flujo se compone de los siguientes procesadores:

1. **GenerateFlowFile**
2. **ExecuteScript**
3. **PublishKafka**
4. **ConsumeKafka**
5. **PutMongo** (dos instancias, una para cada colección).

Cada uno de estos procesadores tendrá entre uno o dos «funnel» (embudos) que serán los encargados de decirnos si el flujo de datos está funcionando correctamente o si en algún proceso no se envían los datos como debería.

4.3. Diseño de la aplicación Web Flask

La aplicación se dividirá en una capa de *front-end* simple hecho en HTML que se encargará de mostrar el mapa del parking y un *back-end* **RESTful** que se encargará de realizar la consulta a MongoDB y devolver los datos. Esta llamada a la base de datos se realizará cada **5 segundos** (Polling) para que los datos mostrados sean lo más cercanos al estado actual de las plazas del parking.

5. Fase de implementación

Durante la fase de implementación detallaremos la instalación y configuración de todas las herramientas y scripts que se utilizarán en el proyecto, siguiendo el diseño establecido en la fase anterior.

5.1. Configuración del entorno

Para la realización de este proyecto, lo primero que se hará será crear una máquina virtual de Lubuntu con la versión 24-04-03 utilizando el programa VirtualBox.

Instalación de Apache NiFi

El primer programa que se instalará será Apache NiFi. Para ello, se requiere la instalación de las versiones de Java necesarias. Las versiones que se utilizarán son la 24 para NiFi y la 11 para Dremio.

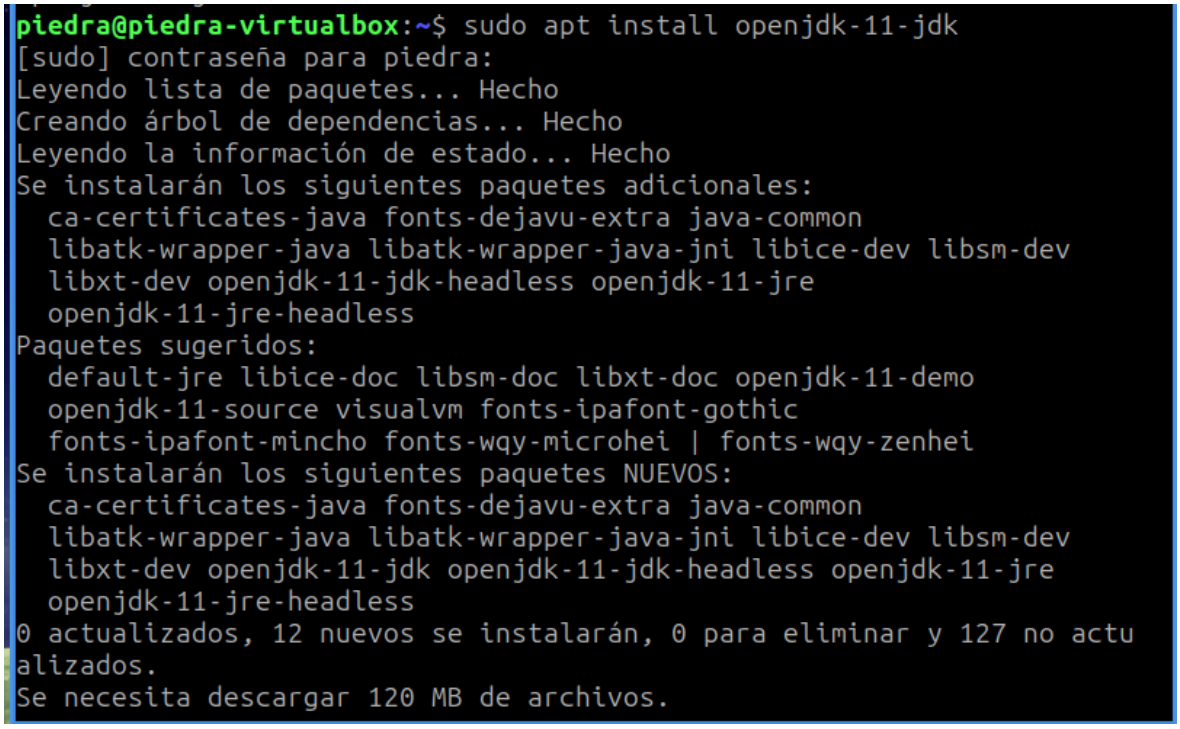


Figura 2: Instalación Java 11

Para que al pache NiFi pueda usar la versión de Java que hemos descargado, tendremos que modificar sus variables de entorno, para ello utilizaremos el comnado: "nano /.bashrcz dentro del archivo colocaremos estas lineas de codigo.

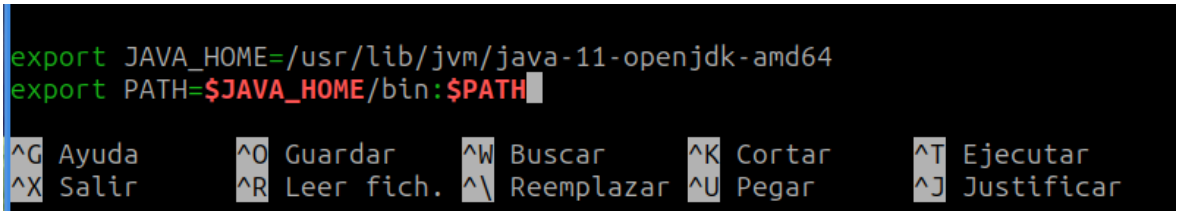


Figura 3: Variables de entorno Java

Cuando añadamos estos comando, ya podremos empezar con la instalación de NiFi, la versión que usaremos será la 2.6.0 y la descargaremos mediante el uso de este comando.



Figura 4: Descarga NiFi

Cuando tengamos el archivo descargado, tendremos que descomprimirlo y moverlo a nuestra carpeta principal de NiFi

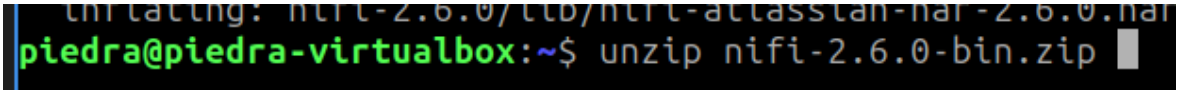


Figura 5: Descompimir descarga NiFi

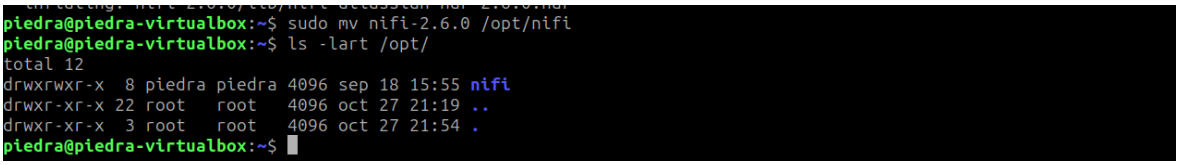


Figura 6: Mover archivos a carpeta NiFi

El siguiente paso para la instalación de NiFi, sera modificar las variables de entorno para que nuestra máquina virtual pueda usar NiFi, esto lo haremos añadiendo estas lineas de código en el mismo archivo donde antes añadimos las variables de entorno de Java.



Figura 7: Variables de entorno NiFi

Ya casi tendriamos el NiFi para poder usar, lo unico que nos faltaria será crer unos credenciales para poder iniciar sesión, lo cual haremos mediante este comando

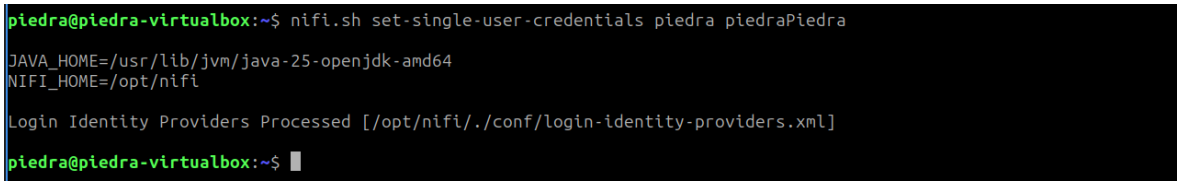


Figura 8: Crear credenciales NiFi

Y ya por último antes de iniciar NiFi tendriamos que comprobar esta bien configurado, estas configuraciones, las encontraremos dentro de la carpeta .ºpt/nifi/confz aqui usaremos el comando nano para revisar el archivo nifi.properties, el codigo dentro de este archivo tendra que tener estos comandos.



Figura 9:

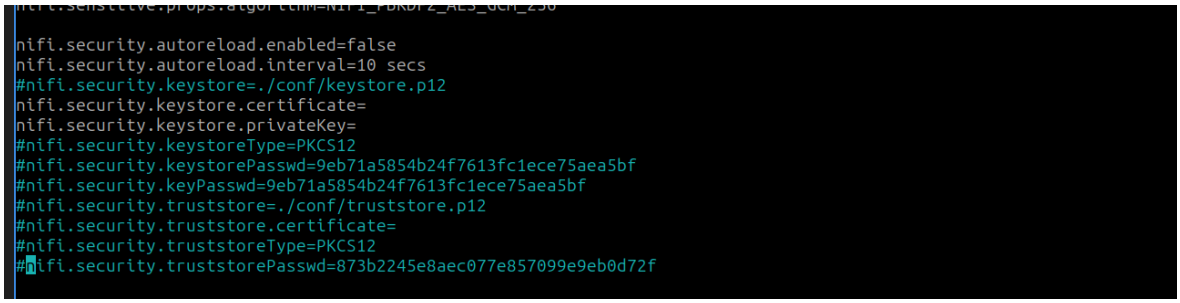


Figura 10:



Figura 11:

Cuando completemos esto, ya podremos inicializar NiFi comenzar a crear nuestro flujo.

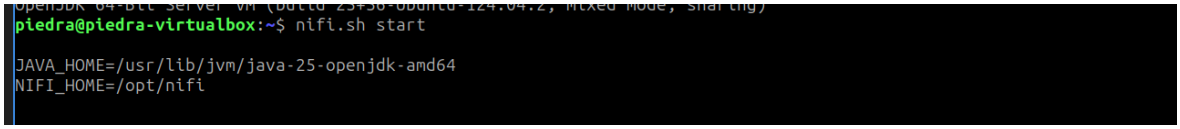


Figura 12: Iniciar NiFi

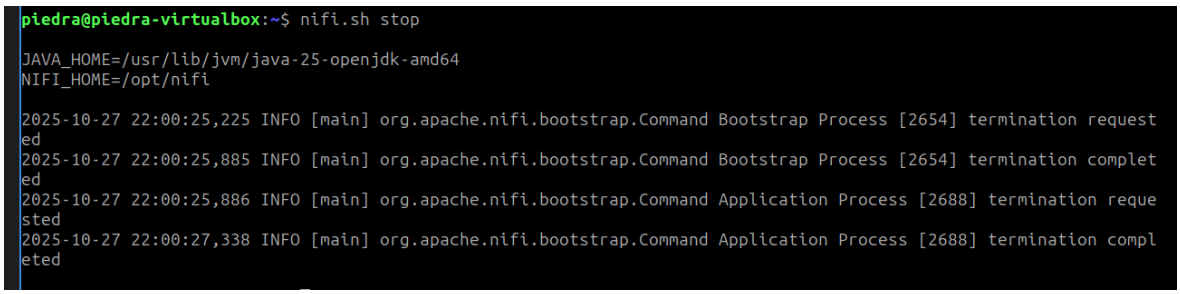


Figura 13: Detener NiFi

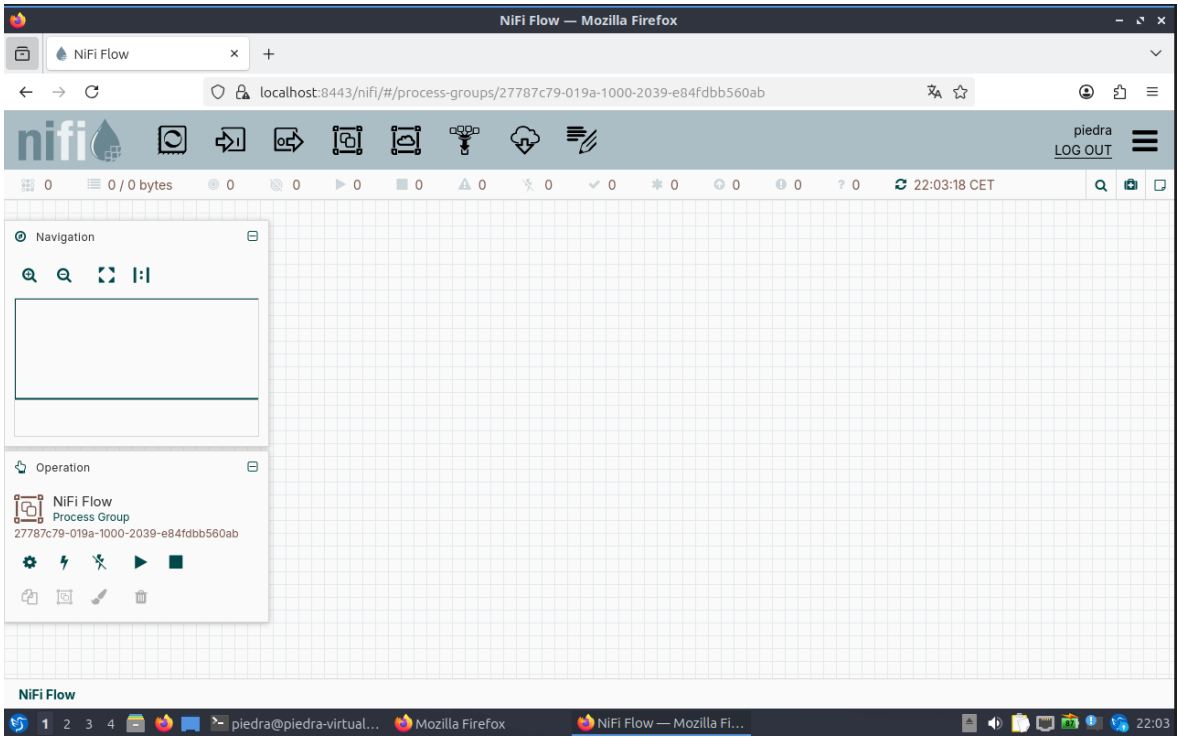


Figura 14: Funcionamiento NiFi

Con todo esto ya tendremos bien configurado y listo para usar NiFi en nuestro proyecto, ahora podremos seguir con el siguiente paso.

5.1.1. Instalación Apache Kafka

La instalación de apache Kafka se realiza de una manera bastante similar a NiFi. Lo primero que haremos sera la descarga de Apache Kafka.

```
piedra@piedra-virtualbox:~$ wget https://dlcdn.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz
--2025-10-27 22:13:26--  https://dlcdn.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz
Resolviendo dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Conectando con dlcdn.apache.org (dlcdn.apache.org)[151.101.2.132]:443... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 133312825 (127M) [application/x-gzip]
Guardando como: 'kafka_2.13-4.1.0.tgz'

kafka_2.13-4.1.0.tgz         100%[=========================================>] 127,14M  45,3MB/s    en 2,8s

2025-10-27 22:13:28 (45,3 MB/s) - 'kafka_2.13-4.1.0.tgz' guardado [133312825/133312825]

piedra@piedra-virtualbox:~$
```

Figura 15: Descarga Apache Kafka

De la misma manera crearemos una carpeta separada para Kafka y mantener un entorno ordenado donde sepamos donde esta cada herramienta.



Figura 16: DescomprimirKafka

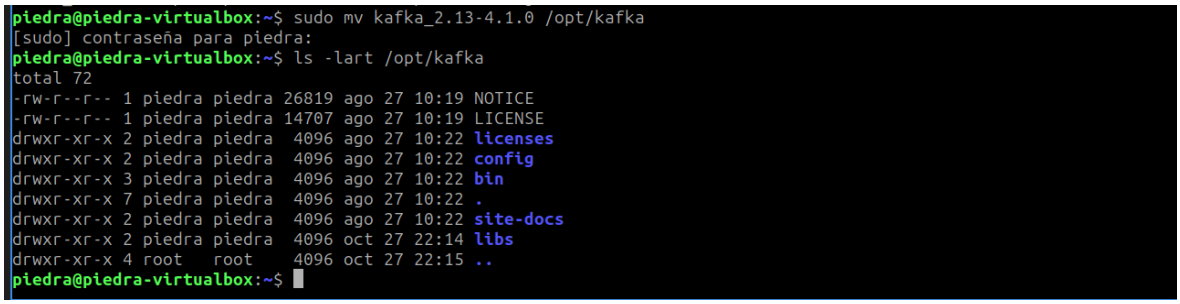


Figura 17: Creación de la carpeta para Kafka

El proximo paso sera añadir las variables de entorno de Kafka a nuestra máquina lo haremos en la misma carpeta donde hemos añadido las otras variables de entorno.



Figura 18: Variables de entorno Kafka

Para nuestro proyecto también sera necesario crear una carpeta Kafka-Datos.

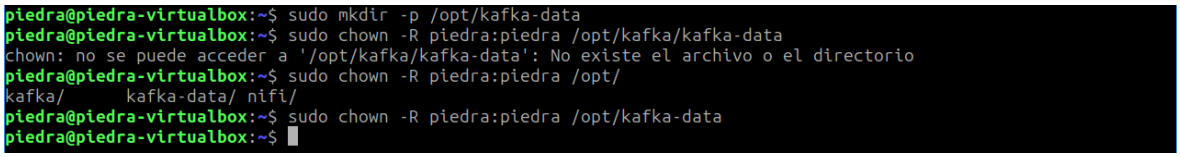


Figura 19: Creación carpeta datos

Lo siguiente que haremos sera generar un ClusterID para que el Kafka que utilicemos para consumir los datos sepa al cluster al que tiene que conectarse.

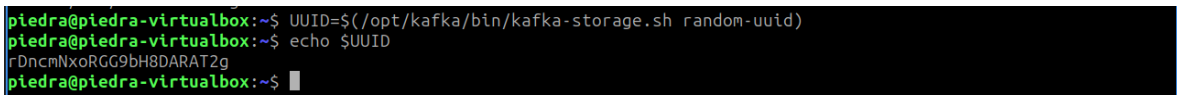


Figura 20: Generación del id del cluster

Para que se puedan guardar los datos bien, tendremos que formatear el almacenamiento para que apunte al cluster que hemos creado con anterioridad de esta manera.

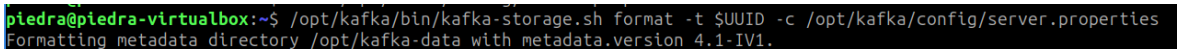


Figura 21: Formatear almacenamiento

El último paso que nos quedaria para poder utilizar Kafka, seria comprobar que tenemos las configuraciones correctas, esto lo haremos utilizando el comando en el archivo server.properties dentro de la carpeta /opt/Kafka/config"

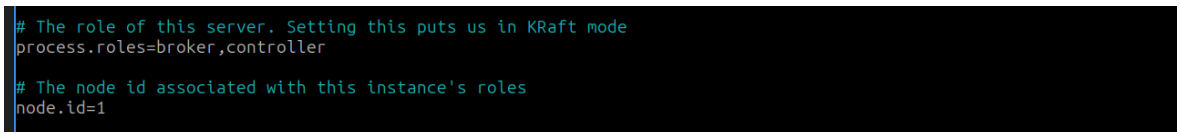


Figura 22: Configuraciones Kafka

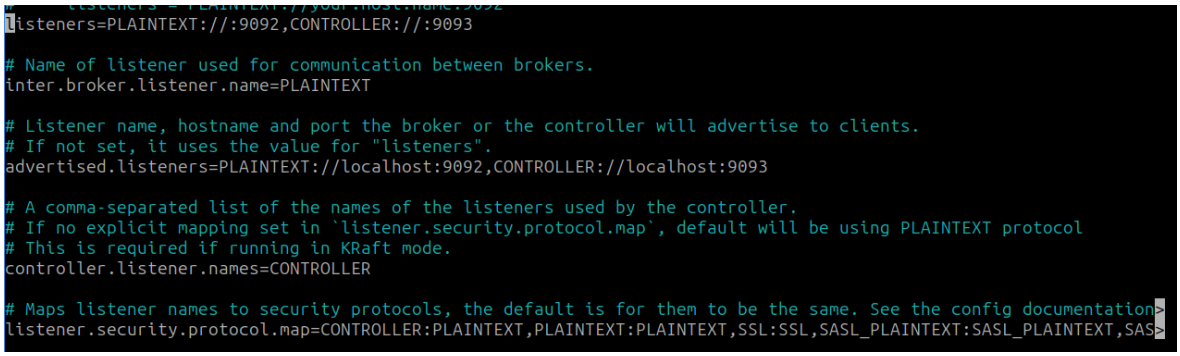


Figura 23: Configuraciones Kafka

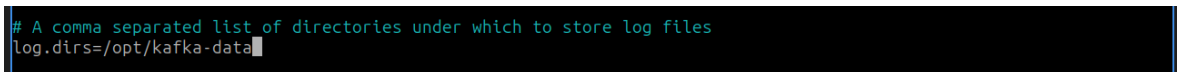


Figura 24: Configuraciones Kafka

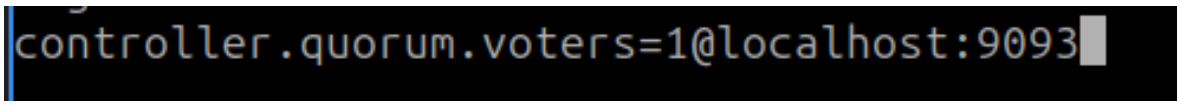


Figura 25: Configuraciones Kafka

Por último haremos una ultima comprobación de la configuración del almacenamiento de datos usando nano en el archivo meta.properties en /opt/kafka-data"



Figura 26: Configuración almacenamiento

Con todas las configuraciones realizadas ya podriamos utilizar Apache Kafka para el manejo de los datos.

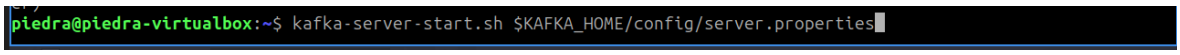


Figura 27: Comando para iniciar Kafka



Figura 28: Comando para detener Kafka

5.1.2. Instalación MongoDB

La siguiente herramienta que tendremos que instalar y configurar será MongoDB que es la base de datos no relacional que utilizaremos en el proyecto.

Importaremos la clave GPG para MongoDB utilizando el siguiente comando.

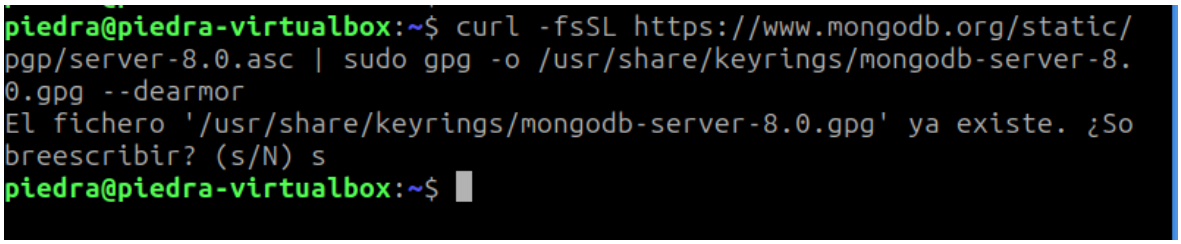


Figura 29: Importación clave GPG

Una vez importada la clave GPG, podemos proceder a crear el archivo de lista de paquetes usando este comando.

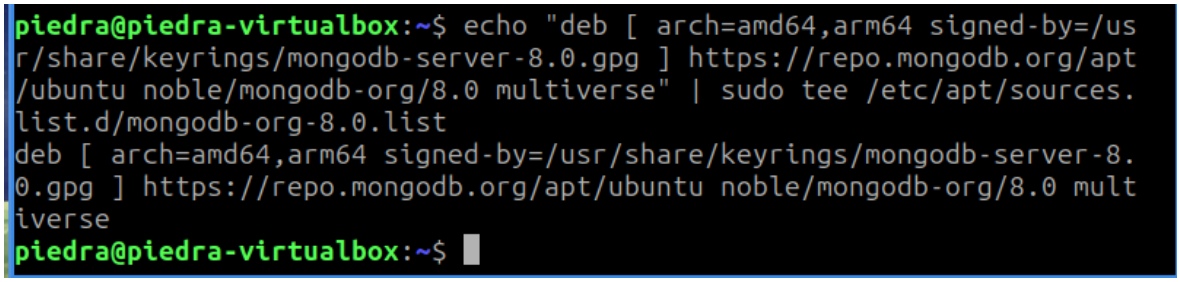


Figura 30: Creación archivo de lista de paquetes

Con estos dos primeros pasos, ya podremos instalar MongoDB utilizando el siguiente comando.

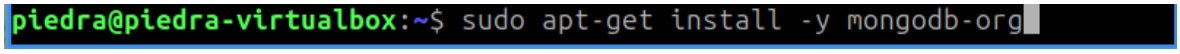


Figura 31: Instalación de MongoDB

Cuando se termine la instalación ya podremos iniciar y utilizar MongoDB con estos dos comandos.

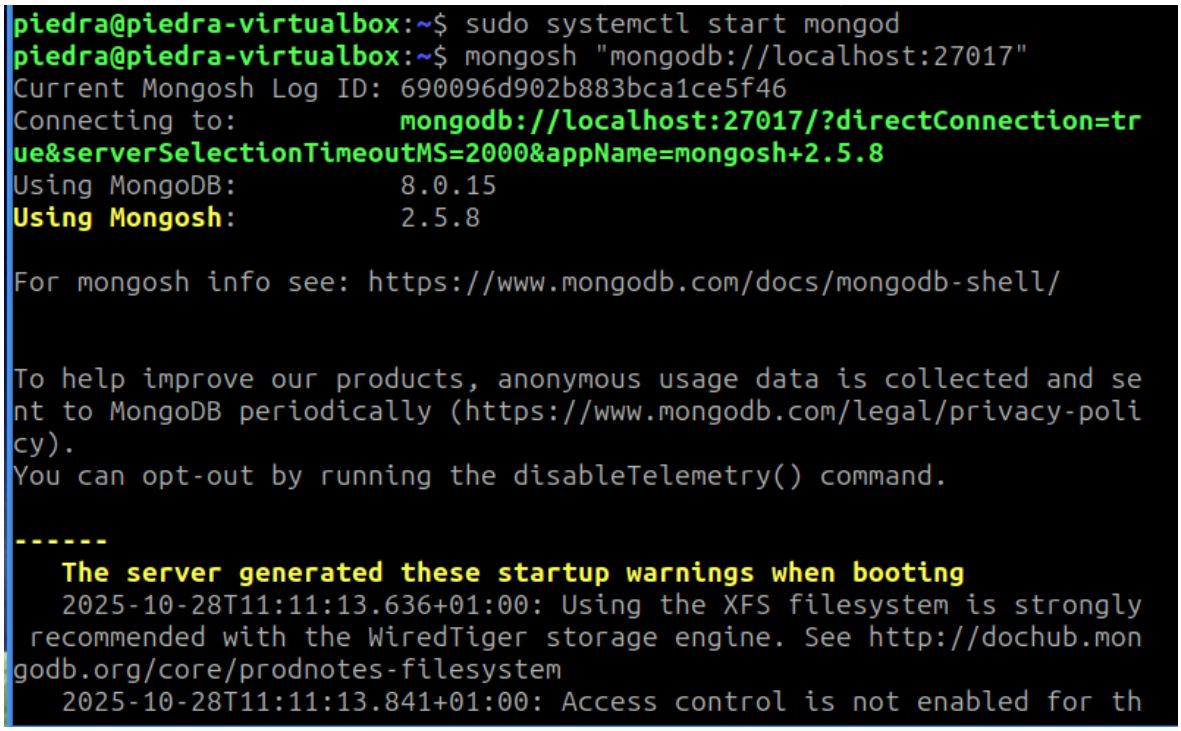


Figura 32: Inicialización de MongoDB

5.1.3. Instalación de Flask

La siguiente herramienta que tendremos que instalar para nuestro proyecto es el Flask.

Flask se trata de un framework de python que permite la creación de interfaces web utilizando html. En este proyecto se utilizará Flask para generar una intefaz web a la que los ciudadanos puedan acceder para revisar el estado de las distintas plazas de los aparcamientos de Cádiz.

Lo primero que tendremos que hacer para instalar Flask, sera comprobar que tengamos instalados Python3 y la instalación del gestor de paquetes de Python que se llama Pip.



Figura 33:



Figura 34:

El siguiente paso sera crear una carpeta para la herramienta de Flask y otra dentro de ella para el proyecto que haremos.



Figura 35: Creación carpeta Flask

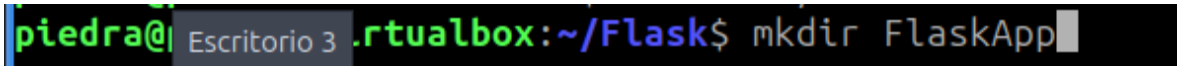


Figura 36: Creación carpeta proyecto

Con esto ya tendríamos la instalación de flask y solo nos faltaría realizar los archivos necesarios para la interfaz web.

El primer paso

5.1.4. Instalación de Dremio

Dremio será la última herramienta que instalaremos y nos permitira poder hacer consultas a las dos bases de datos que tenemos en MongoDB para realizar analisis a largo plazo.

Lo primero que tendremos que hacer antes de la instalación, sera crear un usuario en nuestra máquina y lo configuramos para que no pueda iniciar sesion en la terminal

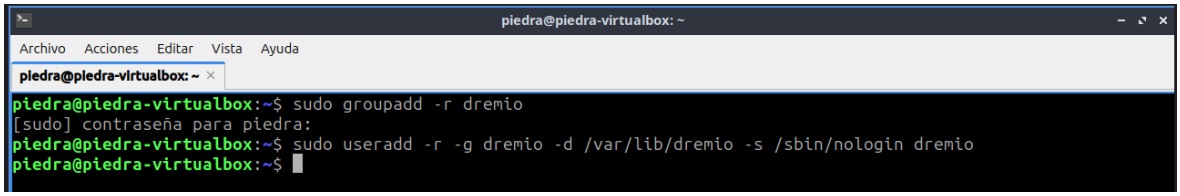


Figura 37: Creación usuario para Dremio

Comprobamos que se haya creado el usuario y tenga la configuración correcta.

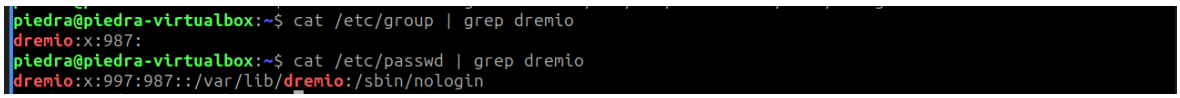


Figura 38: Comprobaciones creación usuario

Con el usuario ya creado, podemos empezar con la descarga de Dremio mediante este comando



Figura 39: Instalación Dremio

Moveremos el archivo que hemos descargado a una nueva carpeta y alli lo descomprimiremos usando este comando:

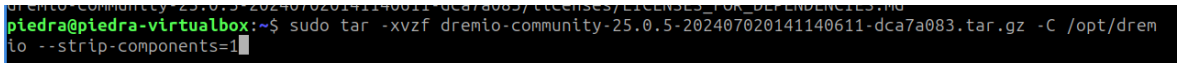


Figura 40: Descomprimir en carpeta

Para usar dremio, necesitaremos darle los permisos necesarios con este comando.



Figura 41: Otorgar permisos a Dremio

Lo último que necesitaremos para poder tener el Dremio activado será crear un enlace simbólico.

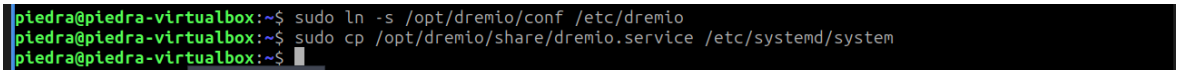


Figura 42: Creación enlace simbólico

Con esto ya habríamos terminado la instalación de Dremio y podríamos utilizarlo ya en nuestro proyecto. Con estos comandos lo iniciaremos y podremos comprobar si se ha iniciado sin problemas.

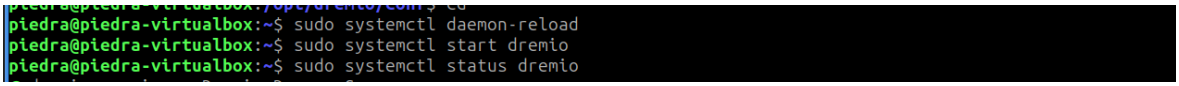


Figura 43:

Con esto ya tendríamos todas las herramientas preparadas y listas para usar en nuestro proyecto.

5.2. Generación del Flujo de Trabajo con Nifi

Con todas las herramientas preparadas, ya podemos empezar con la parte principal del proyecto, el flujo de trabajo. Para realizar este flujo seguiremos las cuatro fases mencionadas en la fase de diseño.

5.2.1. Fase de ingesta de Datos

El flujo para este proyecto lo haremos siguiendo las fases de diseños que hemos comentado con anterioridad en el capitulo 4.

Comenzaremos primeramente por la ingesta de datos, para realizar nuestro proyecto, la idea era utilizar sensores en cada plaza del aparcamiento que nos den los distintos

datos necesarios para el proyecto.Como no tenemos actualmente estos sensores, buscaremos reemplazarlos de alguna manera que los datos que nos lleguen sea lo más fieles a unos sensores de verdad que podamos. Para poder hacer esto NiFi nos puedes ser de mucha utilidad ya que mediante el uso de dos procesos distintos podremos simular la ingesta de datos como si nos vinieran desde los sensores. Los procesos que utilizaremos seran:

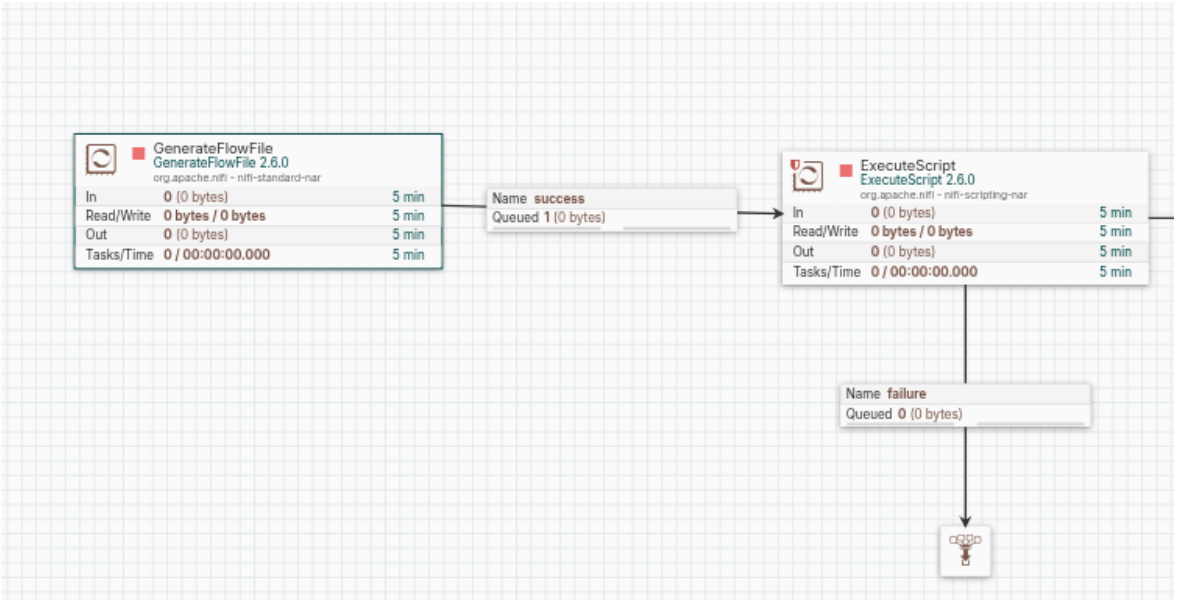


Figura 44: Ingesta de datos

GenerateFlowFile: Este proceso tiene una función muy simple y es que cada 5 segundos genere un nuevo archivo vacio y lo envia al siguiente proceso.

ExecuteScript: Este script es un poco más complejo y es que recibe el fichero vacio que se ha creado con anterioridad y mediante un script que hemos creado en el lenguaje de programación Groovy se rellena con unos datos bastante parecidos a los que nos daria un sensor de verdad si los tuvieramos. El script que utilizamos para generar los datos se encontrará en el apartado 10: Anexos Varios.

5.2.2. Fase de manejo de datos

Durante la fase de manejo de datos se gestionaran los datos simulados que se han creado con anterioridad en los otros procesos y se llevaran hasta las dos bases de datos que vamos a crer para tener un control tanto de los datos en el momento como de un historico para realizar las consultas.

Lo primero que haremos sera crear las bases de datos junto con las colecciones donde iremos añadiendo los datos que nos lleguen a través de los procesos.

La primera base de datos que creemos sera "Parking-historico-db"que tendrá una colección llamada "parking-historico", en esta guardaremos todos los datos que nos lleguen a los sensores. Esta base de datos no se utilizará para la interfaz web sino para realizar consultas a futuro y poder llegar a distintas conclusiones.

```
parking_actual_db> use parking_historico_db
switched to db parking_historico_db
parking_historico_db> db.createCollection("parking")
{ ok: 1 }
parking_historico_db> show dbs
admin                40.00 KiB
config               72.00 KiB
local                72.00 KiB
parking_historico_db  8.00 KiB
```

Figura 45: Creacion BD historico

La segunda base de datos que creemos sera "Parking-actual-db"que tendrá una colección llamada "parking-actual". Esta base de datos funcionara distinto de la anterior que ya que utilizará el patrón de actualización Üpsert". Este patrón lo que hará es comprobar los campos "bay-idz "parking-id"de los datos que llegan nuevos y los comparan con los valores que tenga ya dentro de la base de datos, si ya existen se actualizarán y si no existen se añadirán. Este patrón permite que no existan duplicados y los datos estén lo más actualizados posibles.

```
parking_historico_db> use parking_actual_db
switched to db parking_actual_db
parking_actual_db> db.createCollection("parking_actual")
{ ok: 1 }
parking_actual_db> show dbs
admin                40.00 KiB
config              108.00 KiB
local                72.00 KiB
parking_actual_db     8.00 KiB
parking_historico_db  8.00 KiB
```

Figura 46: Creación BD actual

Con ambas bases de datos ya creadas podemos seguir con el flujo de datos.

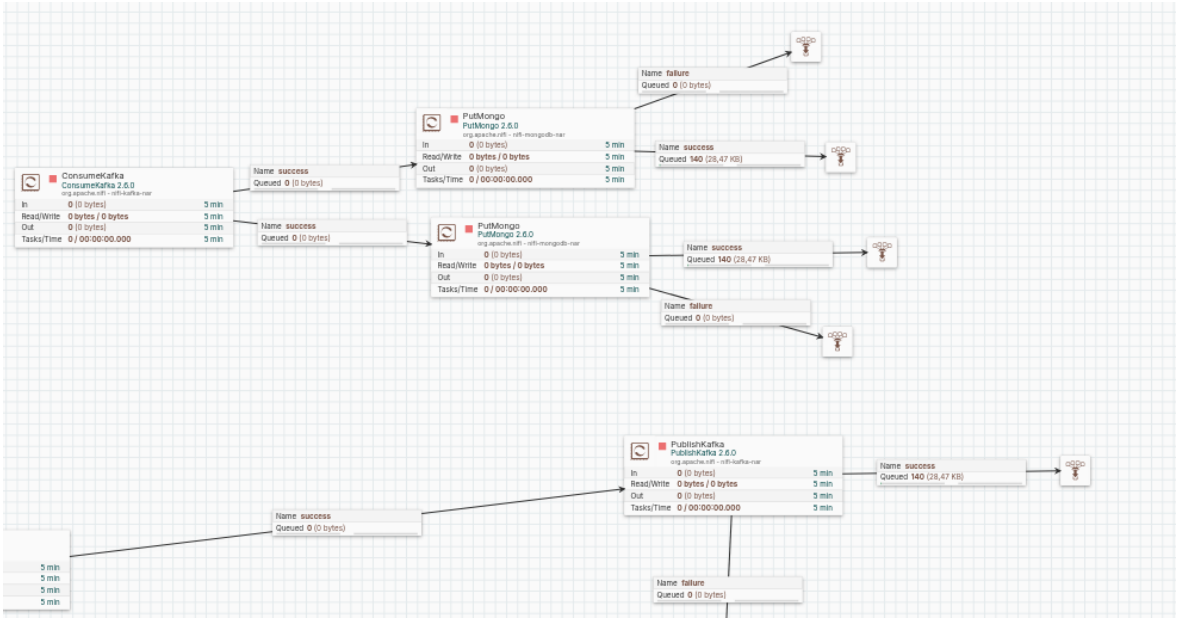


Figura 47: Manejo de datos en el flujo

PublishKafka Este proceso es el que recibe los datos simulados desde el script anterior y los envía a un topic donde seran guardados hasta que sean necesarios. Este proceso será necesario de configurar para evitar que los datos se pierdan.

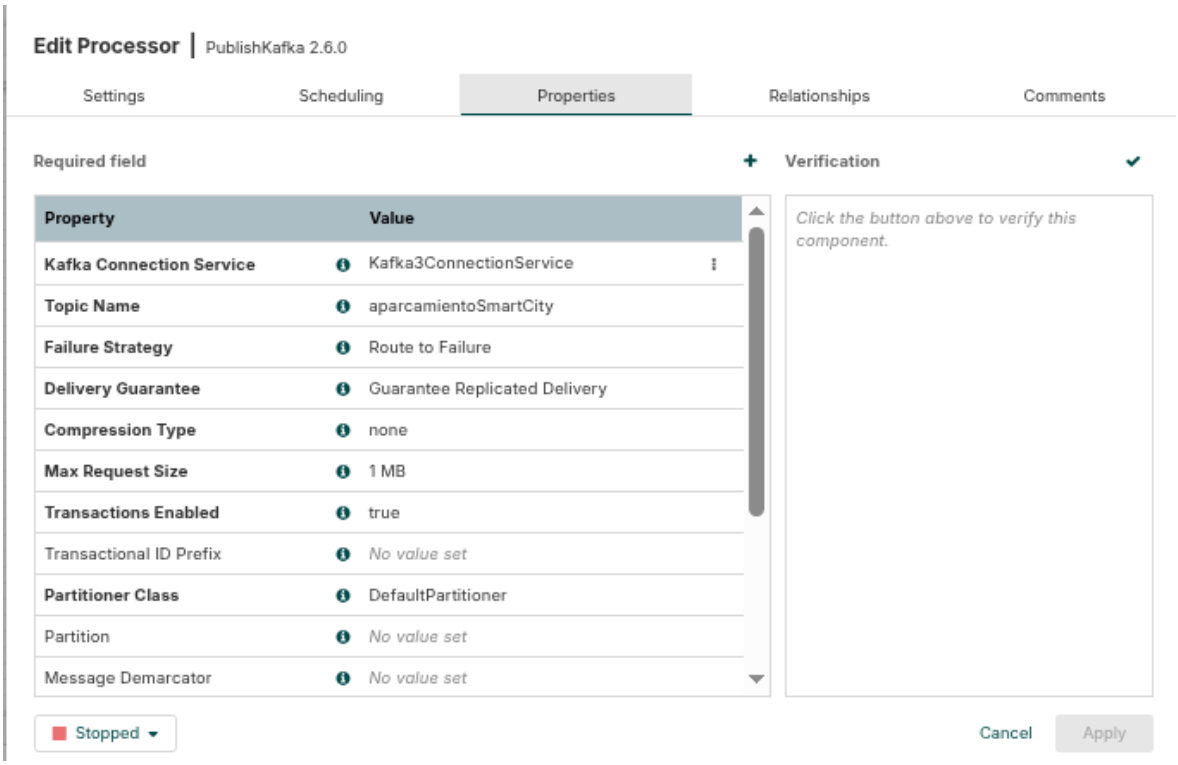
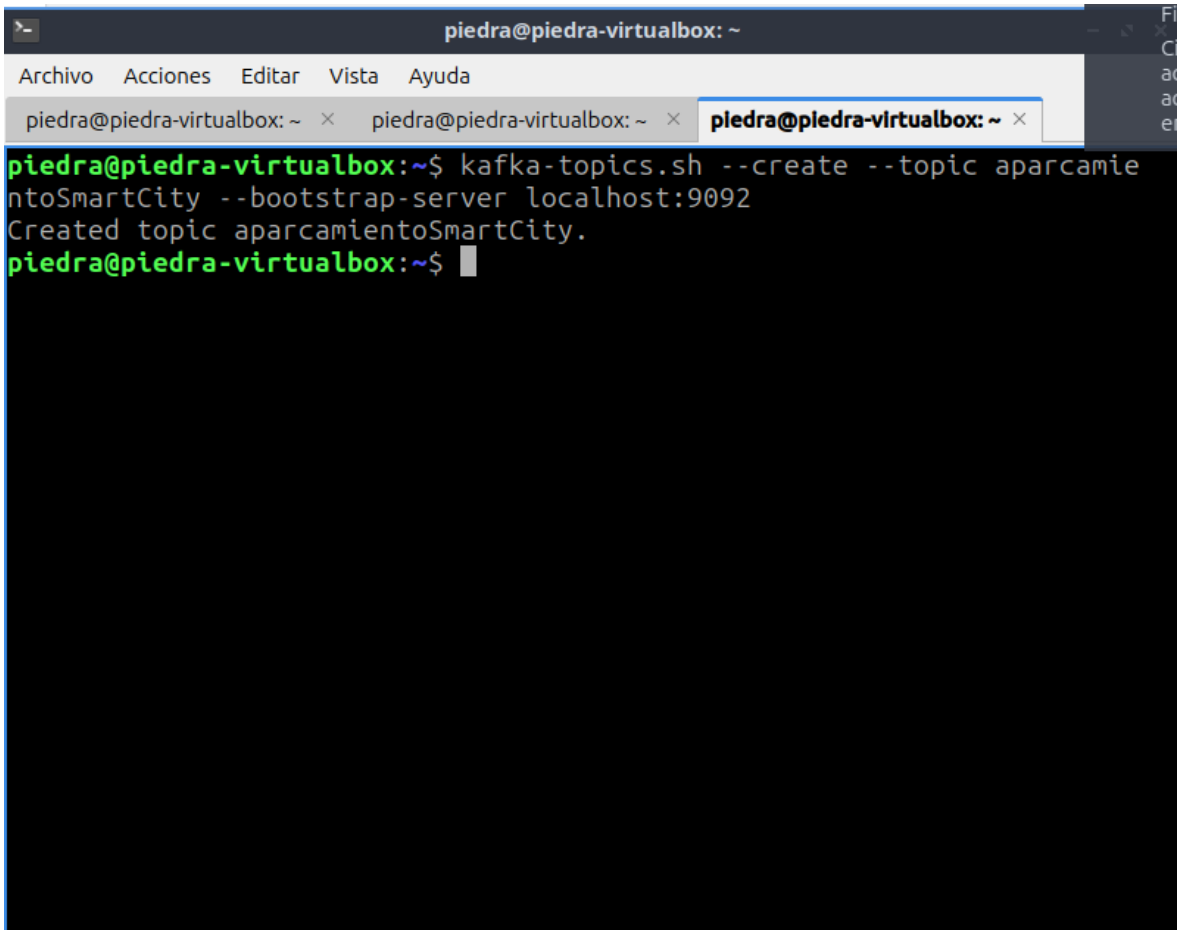


Figura 48: Configuración KafkaPublish

Los campos más importantes y que tendremos que tener en cuenta en este proyecto son los dos primeros:

- **Kafka Connection Service**: Es el encargado de que kafka pueda cumplir su función de manejar los datos, tendremos que añadirlo. Y activarlo en la configuración del servicio.

- **TopicName**: Este campo será donde se guardaran los datos que lleguen al kafka hasta ser utilizados. El topic lo hemos creado con anterioridad mediante este comando



ConsumeKafka Este proceso es el que se encarga de tomar los datos que haya en el topic para luego utilizarlos. Los campos más importantes de este proceso son los siguientes:

- **Kafka Connection Service**: Es el encargado de que kafka pueda cumplir su función de manejar los datos, tendremos que añadirlo. Y activarlo en la configuración del servicio.

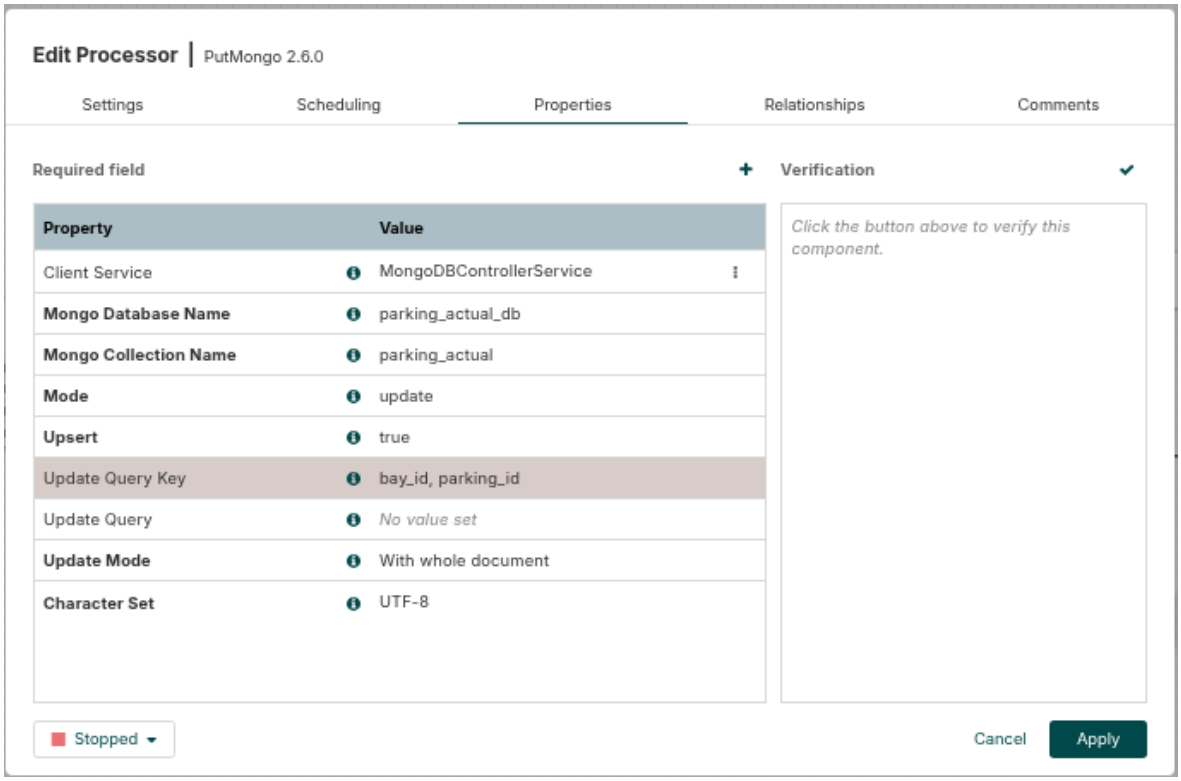


Figura 49: Configuración PutMongo actual

- **TopicName**: Este campo será donde se guardaran los datos que lleguen al kafka hasta ser utilizados. El topic lo hemos creado con anterioridad mediante este comando

PutMongoEs el proceso que toma los datos que les llega y los envia a la base de datos que tenga configurada. En nuestro proyecto utilizaremos dos PutMongo distintos uno para la base de datos actual y otro para la historica, cada uno de ellos con su debida configuración.

Los campos más importantes en la configuración de este proceso son:

- **Client Service** Es el servicio que se encarga de conectar este proceso con la herramienta de MongoDB, para que funcione correctamente también tendremos que cambiar la configuración de este servicio añadiendole nuestro URI de mongodb a las propiedades y habilitandolo posteriormente para que pueda funcionar correctamente.

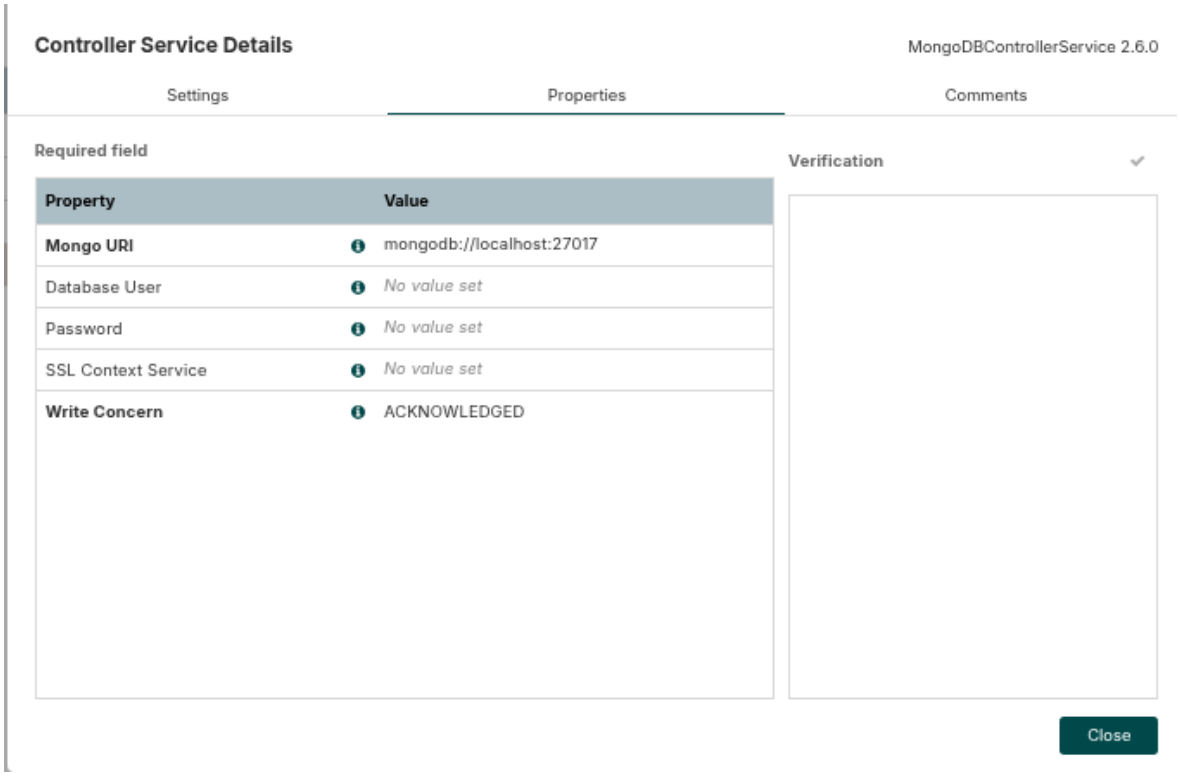


Figura 50: Configuración servicio cliente

- **Mongo Database Name**; Es el nombre de la base de datos a la que va acceder para guardar los datos que le lleguen.
- **Mongo Collection Name**; Es el nombre de la colección a la que va acceder para guardar los datos que le lleguen.
- **Mode**; Es el modo que tomara a la hora de ingresar los datos, puede ser de tipo ingreso o de tipo actualización.
- **Upsert**; Nos dice si la actualización se realizar mediante un upsert no.
- **Update Query Key**; Se trata del valor o valores que se utilizarán para comprobar si ya existe en valor que queremos añadir dentro de nuestra base de datos.

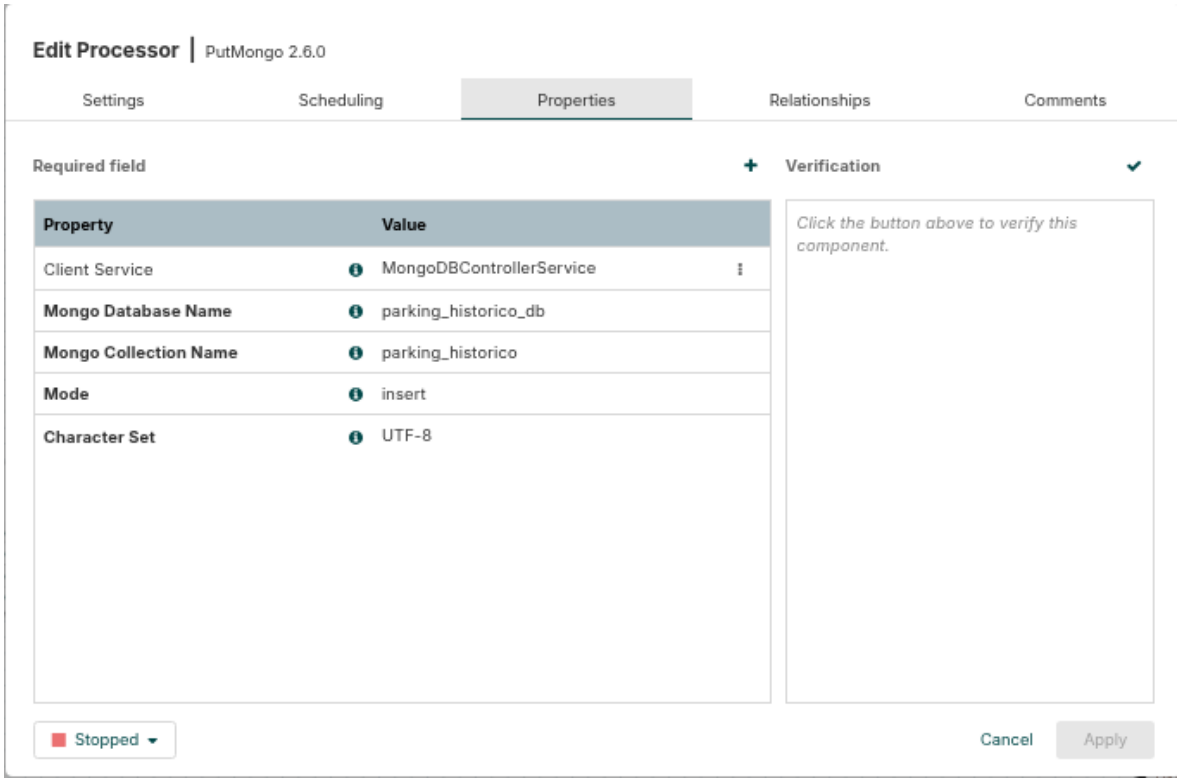


Figura 51: Configuración PutMongo historico

La configuración del PutMongo para la base de datos del historico es bastante similar, lo unico que cambia son la base de datos a la que accede, la colección donde guardará los datos y el modo de añadir los datos, que será un insert ya que en esta base de datos queremos todos los datos que se reciban desde los sensores.

Funnel: Los funnels no son procesos sin embargos son bastante utiles ya que nos permiten supervisar el flujo de datos por si en algun momento hay un fallo, podamos saber donde ha ocurrido e identificarlo con mas facilidad.

5.2.3. Fase de interfaz web

Para la interfaz web usaremos Flask el cual es un framework que utiliza Python y que permite la creación de Interfaces web utilizando HTML. Para este proyecto nuestra interfaz web consistirá de dos archivos distintos: Un back-end llamado SmartParking.py y un front-end llamado index.html que se encuentra dentro de una carpeta llamada templates.

SmartParking.py: Se trata del back-end de nuestra aplicación en el se realizar la conexión con MongoDB para acceder a nuestra base de datos, también añadiremos una función que permita parsear los datos del identificador de la plaza del ayuntamiento para poder ordenarlos correctamente y por último utiliza dos endpoints, uno a la ruta raiz que renderizara el index.html y otro que es el que se encargará de recibir los datos desde la base de datos. Para poder hacer la conexión con la base de datos y

python tendremos que añadir la libreria de PyMongo, la cual instalaremos mediante este comando. El codigo se encontrará en el apartado 10: Anexos varios

```
piedra@piedra-virtualbox:~$ sudo apt install python3-pymongo
```

Figura 52: Instalación PyMongo

Index.html. El index.html es el front-end de nuestra aplicación en el se definirán la vista de nuestra interfaz web. El codigo se encontrará en el apartado 10: Anexos varios. El documento esta dividido en tres partes principales:

- **head** Aqui se encuentra la información del documento, los titulos, estilos, etc...
- **body** Aqui es donde se encuentra el contenido que sera visible en nuestra interfaz web.
- **script** Esta parte contiene el codigo que obtiene los datos dinamicamente desde la base de datos con los registros actuales.

La parte principal de este archivo index.html es el apartado de script que nos permite el acceso a los datos de la base de datos mediante el uso de un fetch(/'data'). Para luego limpiar el contenido que había con anterioridad.

5.2.4. Fase de analisis

En la fase de analisis, utilizaremos los datos que han estado siendo simulados por los procesos de Apache NiFi y guardados en las distintas bases de datos para realizar analisis a futuro. Para realizar estos analisis realizaremos distintas consultas a las bases de datos mediante la herramienta de Dremio..

Para poder realizar estas consultas, necesitaremos conectar nuestras bases de datos con la herramienta de dremio, lo haremos de la sigueinte manera.

Lo primero que haremos será inciar la herramienta de dremio con este comando

```
piedra@piedra-virtualbox:~$ sudo systemctl start dremio
```

Figura 53: Iniciamos Dremio

Después, nos conectaremos a Dremio con los credenciales que hemos creado con anterioridad, comprobando que funciona correctamente y nos envia a esta pantalla.

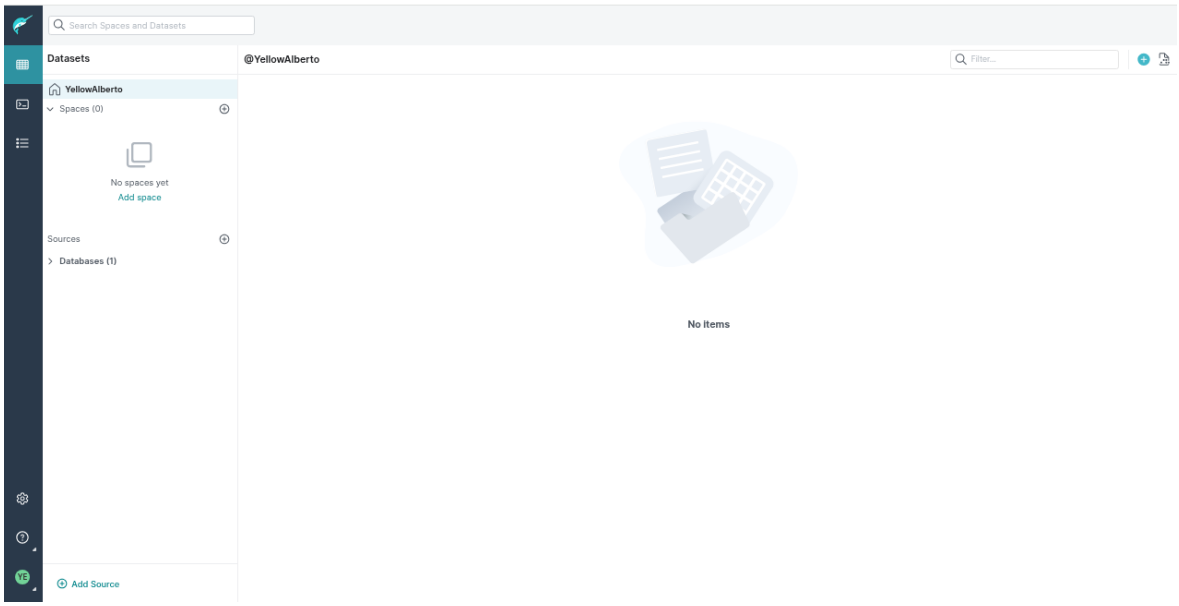


Figura 54: Pantalla inicio Dremio

Ahora tendremos que crear una nueva fuente para realizar la conexión con la base de datos. Esto lo podremos hacer en el botón que se encuentra a la izquierda de la pantalla.

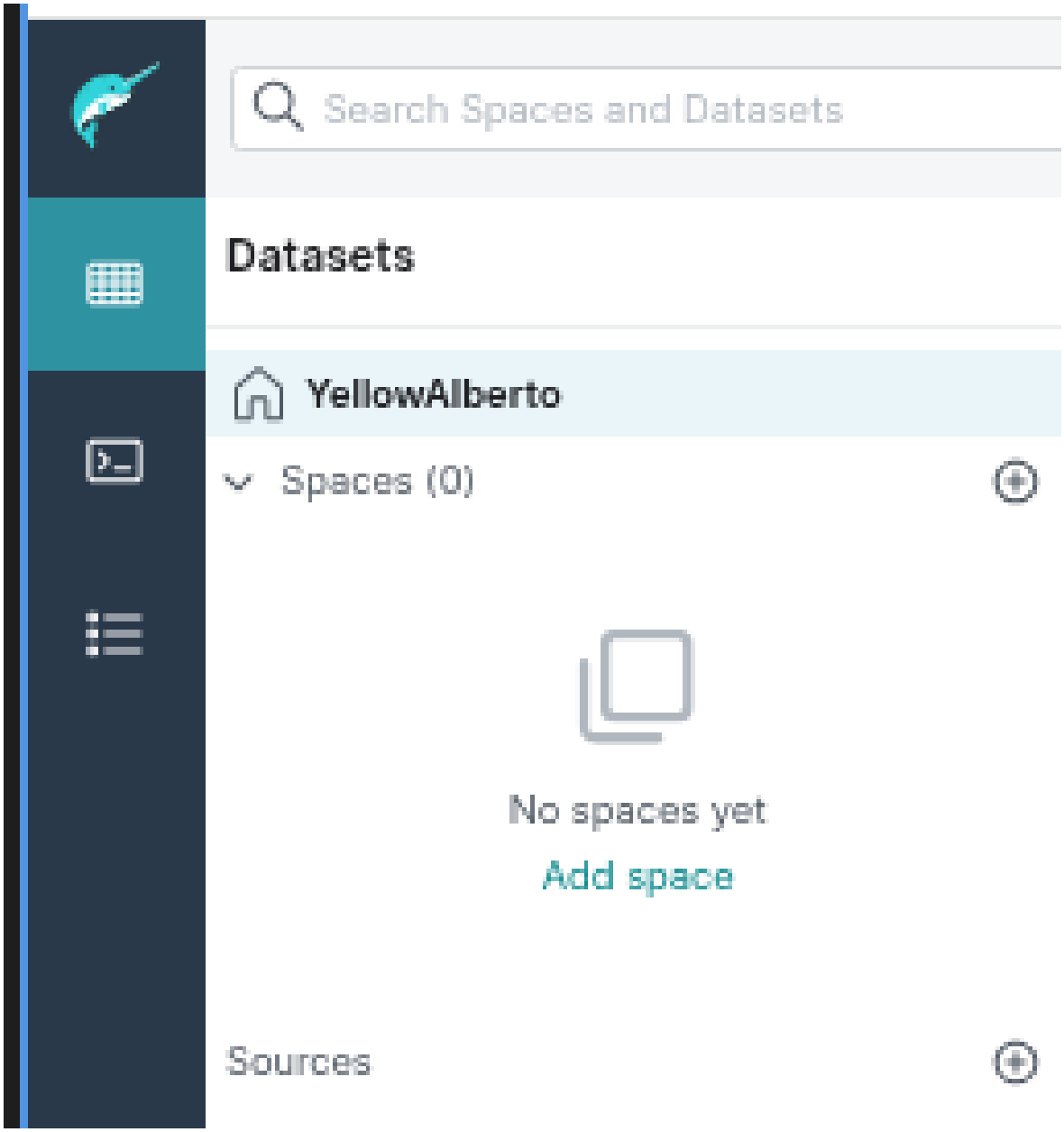


Figura 55: Insertar nueva Fuente

Añadiremos la fuente de MongoDB.
Crearemos la fuente con la configuración de nuestra base de datos

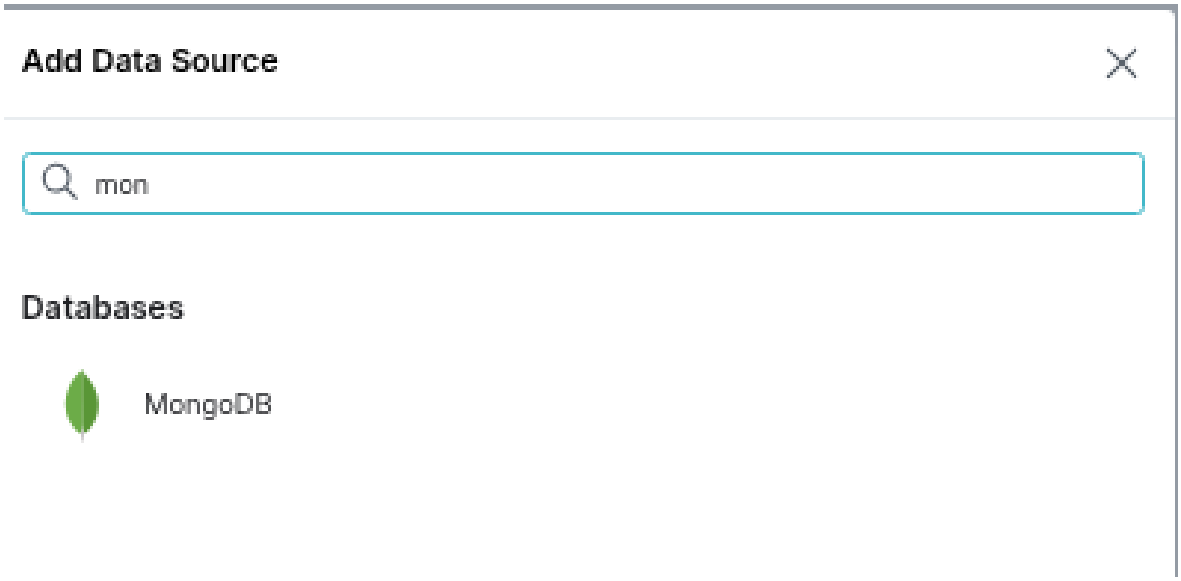


Figura 56: Añadimos la fuente de MongoDB

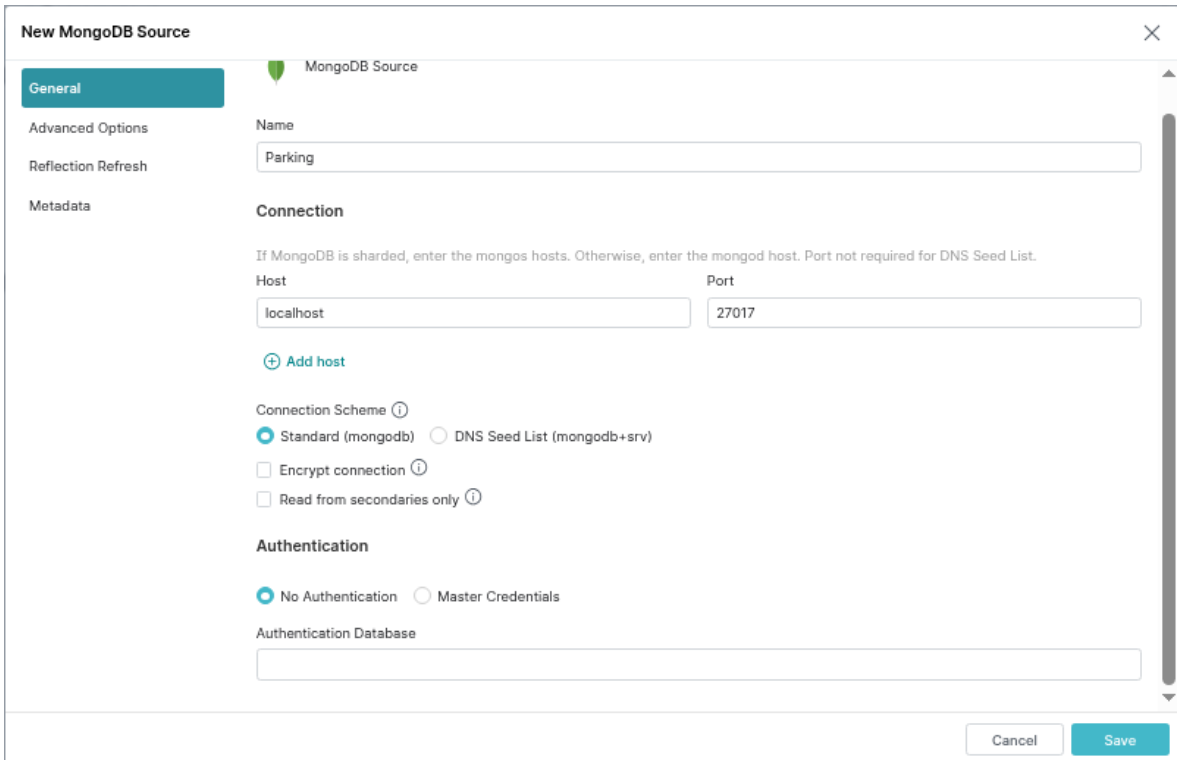


Figura 57: Configuramos la fuente de la BD

Con esto ya tendriamos la base de datos conectada a dremio y podriamos realizar las consultas. Para este proyecto, realizaremos 5 consultas distintas que nos daran mucha información acerca del proyecto y de los propios aparcamientos:

- La primera consulta que haremos será para realizar un conteo del historico de las plazas de parking.

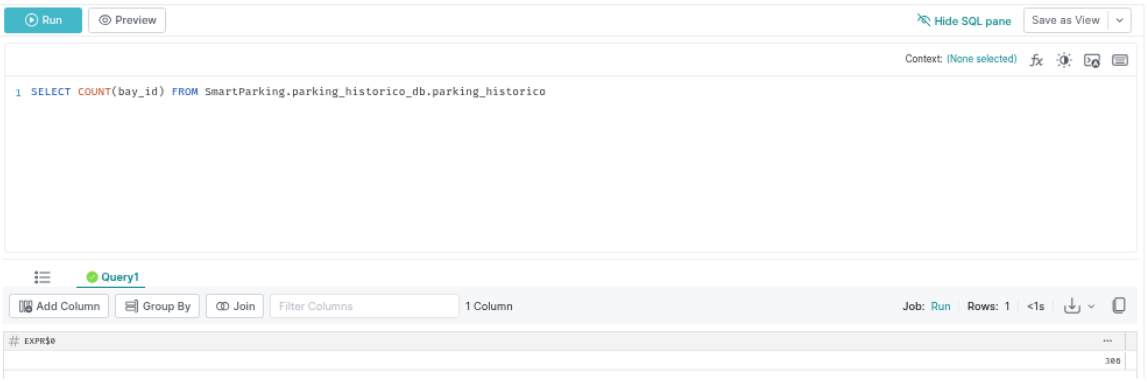


Figura 58: Primera Consulta

- La segunda consulta que haremos será para comprobar el estado de las baterías de nuestros sensores en caso de que alguna este demasiado baja.

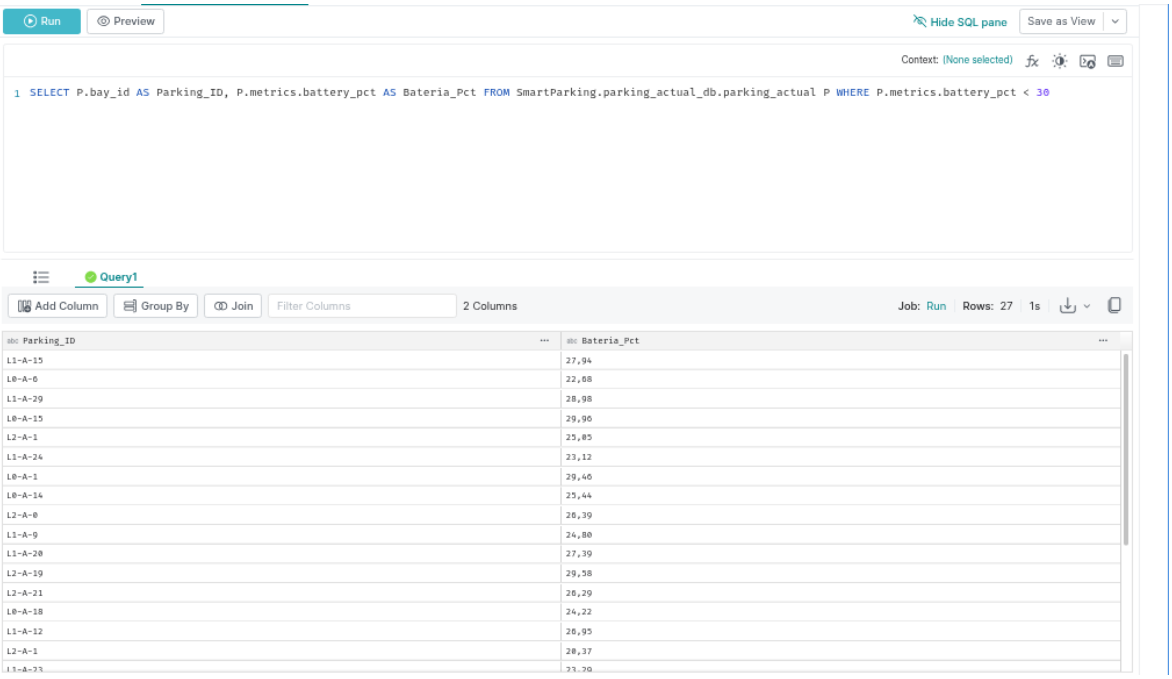


Figura 59: Segunda Consulta

- La tercera consulta que realizaremos es un poco más complejo pero nos aporta más información, y es que nos permite saber cuantas plazas de aparcamiento se ocupan segun el periodo horario, siendo este dividido en 3. Mañana(8AM-2PM), tarde(2PM-10PM) y noche(10PM-8AM).

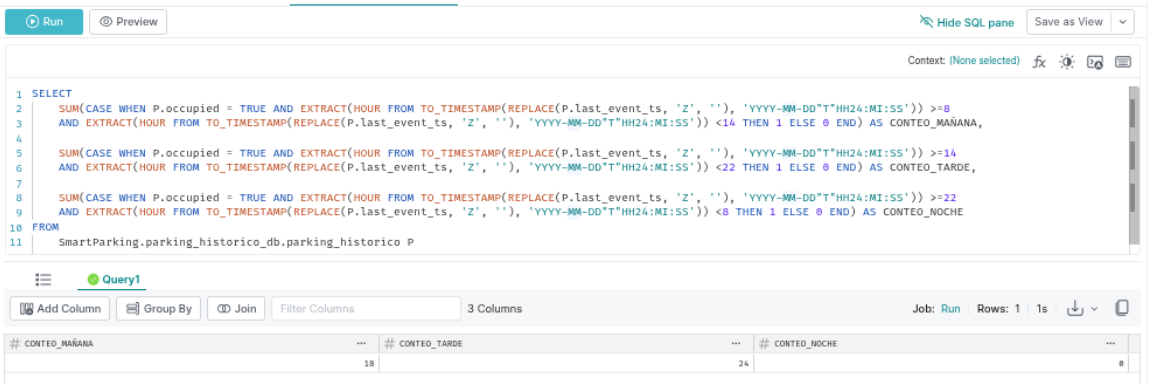


Figura 60: Tercera consulta

- La cuarta consulta nos permitirá saber la temperatura máxima, mínima y media de todas las plazas del parking a tiempo real.



Figura 61: Cuarta consulta

- La quinta consulta nos permite saber cuantas plazas del parking habia ocupadas en cada nivel dependiendo de la hora. Con esta consulta y la tercera consulta podremos analizar que momentos del día y que pisas son los más ocupados.

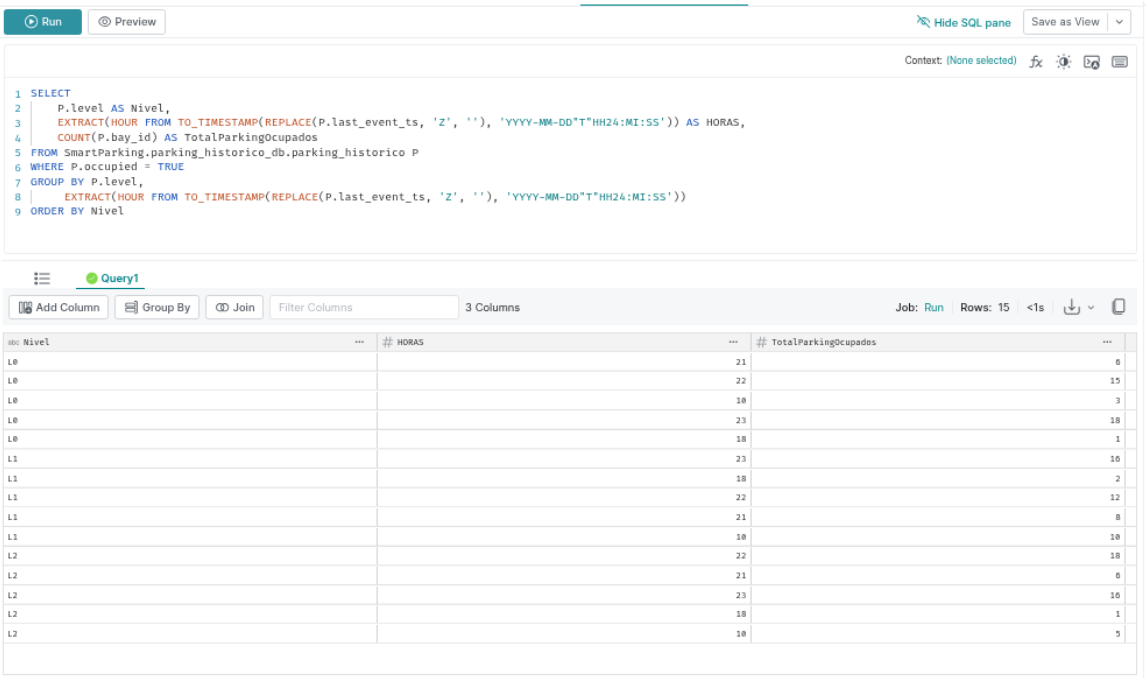


Figura 62: Quinta consulta

Con esto ya habriamos completado la fase de implementación y solo nos quedaría comprobar que funcione todo correctamente en la fase de pruebas.

6. Fase de pruebas

Para comprobar que la implementación que hemos realizado para nuestro proyecto funciona correctamente, vamos a hacer un seguimiento del flujo de nuestros datos hasta que llegan a la página web.

Para ello lo primero que haremos sea quitar todos los datos que haya ya en nuestra base de datos actual para que podamos comprobar que los datos sean todos nuevos, lo haremos mediante esos comandos.

```
parking_actual_db> use parking_actual_db
already on db parking_actual_db
parking_actual_db> db.parking_actual.deleteMany({})
{ acknowledged: true, deletedCount: 141 }
```

Figura 63: Eliminamos los valores antiguos

También eliminaremos todos los datos que se encuentren en la cola de NiFi, para que todos los datos que nos lleguen sean nuevos.

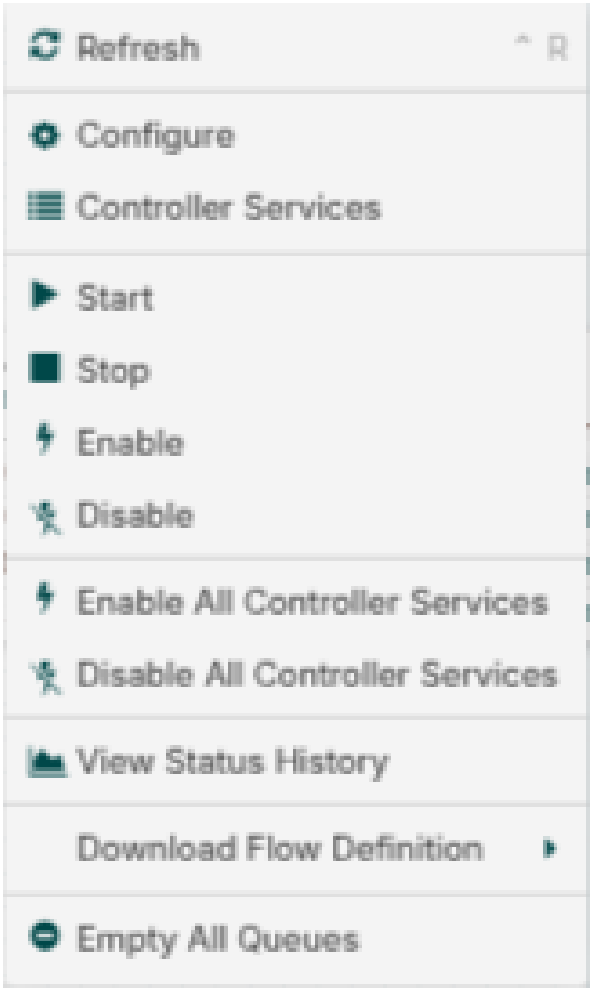


Figura 64: Vaciamos las colas del flujo

Con todos los valores antiguos eliminados y las colas vacias ya podemos comprobar si funciona correctamente y añade los nuevos datos de los sensores.

En la herramienta de NiFi activaremos el flujo para que comience a generar datos pulsando en el botón del "play.ª la izquierda del flujo

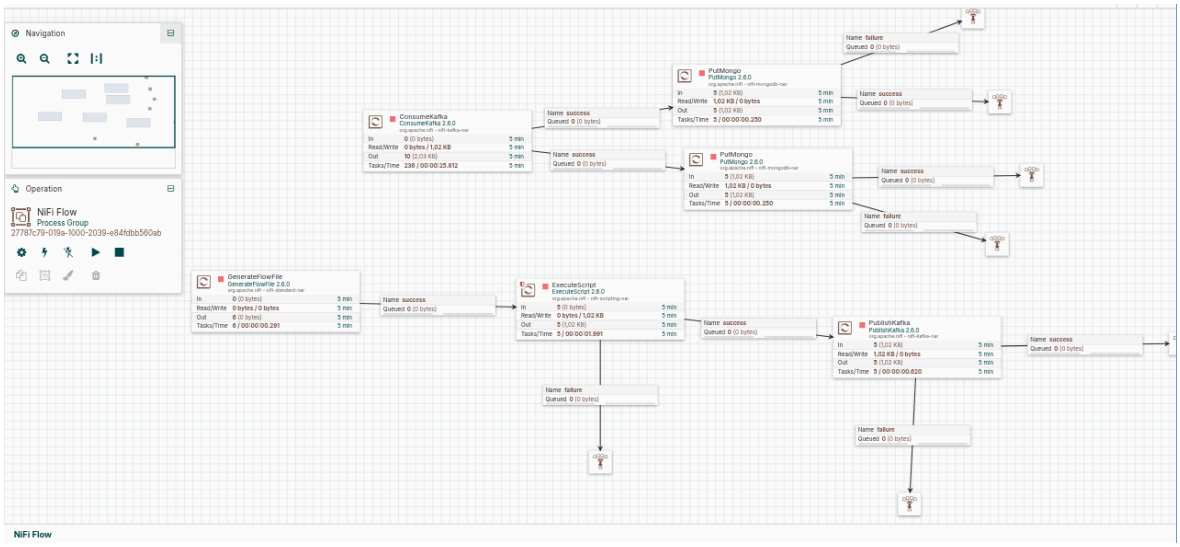


Figura 65: Activamos el flujo

Con el flujo encendido, ya podremos comprobar que funciona todo correctamente de tres manerass:

[H]

- Comprobando las conexiónes de los procesos que los datos sean exitososy ninguno falle
- Comprobamos la interfaz web, si con el flujo activado cambia el contenido que muestra es que funciona todo correctamente.

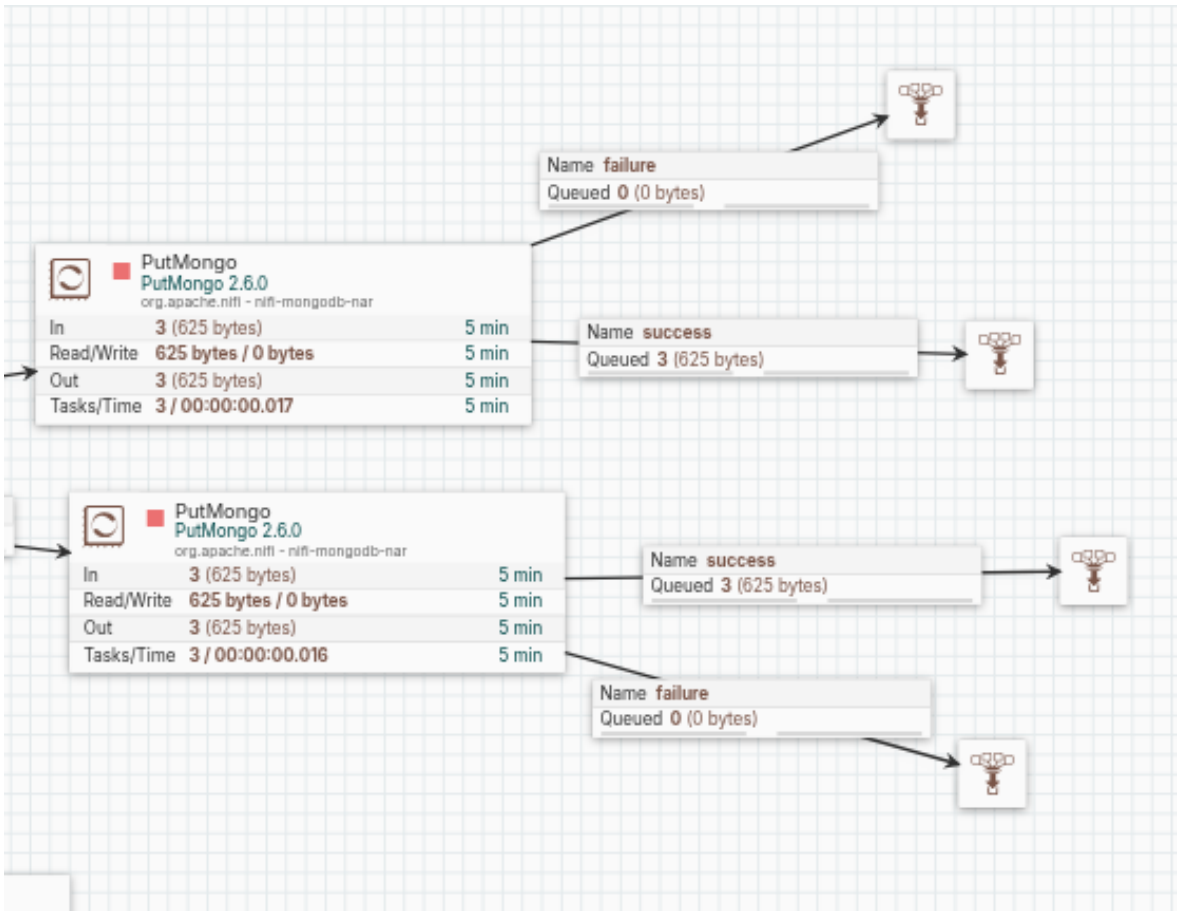


Figura 66: Comprobamos las conexiones de los procesos



Figura 67: Comprobamos que se añadan valores



Figura 68: Se han añadido nuevos valores

- La última manera que tenemos para comprobar que el flujo funciona correctamente es realizar consultas a la base de datos utilizando Dremio para asegurarnos de que los datos llegan correctamente a la base de datos.

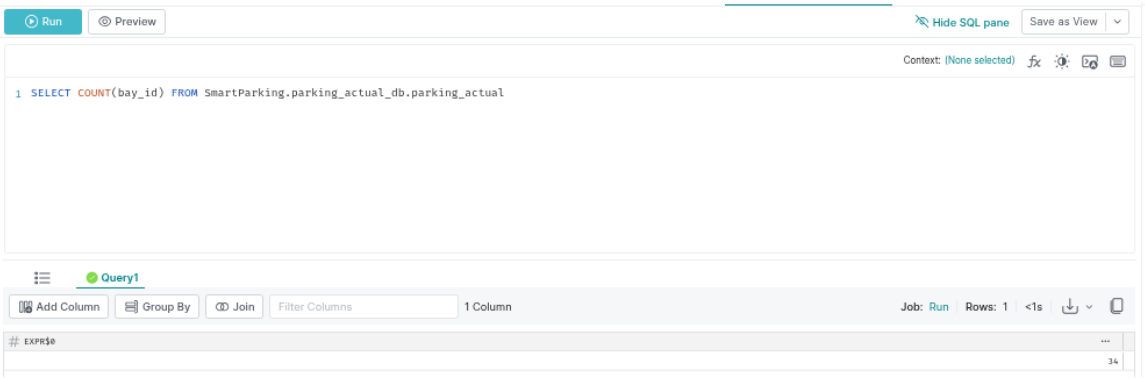


Figura 69: Comprobación mediante consulta

7. Conclusiones y líneas futuras

7.1. Conclusiones

El proyecto **SmartParkingFlow** ha culminado con la implementación exitosa de una arquitectura lakehouse distribuida y escalable para la gestión de datos de un aparcamiento inteligente. Se han alcanzado todos los objetivos iniciales, demostrando la viabilidad de un sistema que ofrece información en tiempo real a los ciudadanos y capacidades de análisis profundas para la administración.

Logros principales:

- **Arquitectura de Flujo Robusta** (NiFi y Kafka): Se logró establecer una tubería de datos eficiente utilizando Apache NiFi para la simulación, ingesta y división del flujo. La integración con Apache Kafka garantizó la deacoplamiento del sistema, permitiendo que la información fluya de manera fiable y resiliente entre la ingesta y el almacenamiento.
- **Doble Persistencia Inteligente** (MongoDB): El diseño de la persistencia fue clave. El uso de MongoDB y el patrón de Upsert en la colección Parking-actual-db aseguró que la interfaz web siempre acceda al estado más reciente de cada plaza sin redundancia. Paralelamente, la colección Parking-historico-db se estableció como un repositorio invaluable para el análisis a largo plazo.
- **Visualización Orientada al Ciudadano** (Flask): La aplicación Flask cumplió con el requisito de ofrecer una visualización simple e intuitiva del estado del parking en çasi tiempo real". La técnica de Polling implementada asegura que el usuario disponga de datos actualizados cada 5 segundos, mejorando la experiencia de estacionamiento en la ciudad.
- **Capacidad de Análisis Avanzado** (Dremio): La conexión entre Dremio y las colecciones de MongoDB demostró la capacidad del sistema para transformar datos NoSQL en información estructurada mediante consultas descriptivas. Las cinco consultas diseñadas validan la posibilidad de extraer conclusiones sobre el uso, la ocupación por hora/nivel y el estado de los sensores (batería, temperatura), cumpliendo el requisito de soporte a la toma de decisiones.

En resumen, SmartParking Flow permiten solucionar no solo solucionar la necesidad inmediata de la información en relacionada a los aparcamientos en Cádiz sino que también esta perfectamente preparado para el crecimiento, ya que el proyecto puede adaptarse facilmente a nuevos sensores o tipos de datos con facilidad.

7.2. Lineas futúras

El proyecto de SmartParkingFlow podrá ser mejorado en un futuro mediante distintas maneras como:

7.2.1. Evolución de la ingesta de datos

- **Integración con hardware real** En un futuro, se podria reemplazar el proceso que simula los datos de los sensores y utilizar sensores reales, aunque necesitará añadir validación de los datos y algunos cambios en el flujo de NiFi.
- **Validación** Añadir en nuestro flujo de datos de NiFi distintos procesos que se encarguen e garantizar la calidad de los datos. Además se podrian añadir más información a los mensajes para añadir más contesxtos y poder realizar un mejor analisis.

7.2.2. Optimización de la interfaz web

- **Comunicación en tiempo real** Se podría cambiar el mecanismo de polling de Flask por la utilización de WebSockets que permitirian que las actualizaciones de la web sean instantaneas al momento de que haya un cambio de estado en los aparcamientos.
- **Mejora de la experiencia de usuario** Se podrían implementar funciones de geolocalización o importar mapas interactivas para que muestren los parkings más cercanos al ciudadano en ese momento y su estado.

7.2.3. Machine learning y analisis predictivo

- **Modelo de predicción de ocupación** Se podría utilizar la base de datos que contiene los historicos para entrenar modelos de machine learning para realizar predicciones de las ocupaciones del parking con anterioridad y mostrar la información para que los ciudadanos puedan hacerse una idea de si habrá plazas de parkings vacias.
- **Detecciones de anomalias en sensores** Se podrían desarrollar modelos que permitan detectar anomalias en los datos que nos llegan de los sensores para poder realizar un mantenimiento y evitar que los sensores se estropeen.

8. Bibliografía

- Instalación de MongoDB en la máquina virtual:

  `https://www.datacamp.com/es/tutorial/install-mongodb-on-ubuntu`

- Revisión de comandos para la generación y utilización de las colecciones:

  `https://www.mongodb.com/docs/manual/core/databases-and-collections/`

- Instalación y utilización de Flask en la máquina virtual:

  `https://www.youtube.com/watch?v=82xc4hnZQeU`

- Información relativa a Kafka

  `https://www.redhat.com/es/topics/integration/what-is-apache-kafka`

- Información de Flask

  `https://www.arsys.es/blog/que-es-flask-python-y-cuales-son-sus-ventajas`

Utilización de inteligencia artificial para la creación del script de simulación de los sensores y resolución de dudas.

9. Diario de trabajo

- **27-10-2025**: Comienzo del proyecto, realización de la parte de la introducción e instalación de dos de los programas necesarios: Apache NiFi y Apache Kafka.
- **28-10-2025**: Instalación de MongoDB.
- **02-11-2025**: Creación base de datos de MongoDB (Actual e histórico) y creación de flujo en Nifi utilizando KAFKA para enviar los datos creados en JSON a ambas bases de datos. Instalación de Flask.
- **03-11-2025**: Creación de la página web mediante Flask para poder mostrar los estados de los aparcamientos en el momento y con los datos que se actualizan cada 5 segundos.
- **04-11-2025**: Instalación Dremio.
- **05-11-2025**: Creación de las cinco consultas para realizar los análisis.
- **06-11-2025**: Realización de la memoria del proyecto
- **07-11-2025**: Realización de la memoria del proyecto
- **08-11-2025**: Realización de la memoria del proyecto

10. Anexos varios

Proceso Execute Script

```
def flowFile = session.get()
if (flowFile != null) {
    try {
        def now = new Date()
        def ultimaFecha = new Date(now.time - 1000)

        def level = "L" + (Math.random() * 3).toInteger()  // sin decimales
        def parkingID = "PK-CADIZ-" + (Math.random() * 3).toInteger()  // sin decimales
        def bayID = level + "-A-" + (Math.random() * 30).toInteger()  // sin decimales

        SimpleDateFormat isoFormat = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss'Z'")
        def formattedDate = isoFormat.format(now)
        def formattedUltimaFecha = isoFormat.format(ultimaFecha)

        // Formato para los valores con 2 decimales
        def decimalFormat = new DecimalFormat("#.00")

        def data = [
            bay_id: bayID,
            parking_id: parkingID,
            level: level,
            occupied: Math.random() < 0.5,
            last_event_ts: formattedUltimaFecha,
            metrics: [
                temperature_c: decimalFormat.format(20 + Math.random() * 10), // 2 decimales
                battery_pct: decimalFormat.format(Math.random() * 80 + 20) // 2 decimales
            ],
            updated_at: formattedDate
        ]

        flowFile = session.write(flowFile) { outputStream ->
            outputStream.write(new groovy.json.JsonBuilder(data).toString().getBytes())
        }

        flowFile = session.putAttribute(flowFile, "mime.type", "application/json")

        session.transfer(flowFile, REL_SUCCESS)

    } catch (Exception e) {
        log.error("Error al procesar el flowFile: ", e)
        session.transfer(flowFile, REL_FAILURE)
    }
} else {
    log.warn("No se recibió un flowFile")
}
```

Figura 70: Script generación de datos

Este código es el encargado de generar los datos simulando los datos que nos daría un sensor real. Para este codigo utilizaremos un lenguaje de programación llamado Groovy esta basando en Java pero tiene bastantes caracteristicas similares con otros lenguajes como Python, Ruby, etc..

El codigo lo primero que hará sera recibir el archivo vacio que se genera con anterioridad en el flujo de NiFi y de ahi generará los datos de los sensores de manera aleatoria, por último lo enviará de nuevo al flujo pero con los datos ya generados.

```python
from flask import Flask, render_template, jsonify
from pymongo import MongoClient
import re

app = Flask(__name__)

client = MongoClient("mongodb://localhost:27017/")
db = client.parking_actual_db
collection = db.parking_actual

def parse_bay_id(bay_id):
    parts = re.split(r'(\d+)', bay_id)
    return [int(part) if part.isdigit() else part for part in parts]

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/data')
def get_parking_data():
    parking_data = list(collection.find())
    parking_data.sort(key=lambda x: parse_bay_id(x.get('bay_id', '')))

    parking_status = []
    for item in parking_data:
        metrics = item.get('metrics') or {}
        temperature = metrics.get('temperature_c') or metrics.get('temperature') or 'N/A'
        battery = metrics.get('battery_pct') or metrics.get('battery') or 'N/A'

        parking_status.append({
            'bay_id': item.get('bay_id', ''),
            'parking_id': item.get('parking_id', ''),
            'level': item.get('level', ''),
            'occupied': item.get('occupied', False),
            'last_event_ts': item.get('last_event_ts', ''),
            'temperature_c': temperature,
            'battery_pct': battery,
            'color': 'green' if not item.get('occupied', False) else 'red'
        })

    return jsonify(parking_status)

if __name__ == '__main__':
    app.run(debug=True)
```

Figura 71: Codigo SmartParking.py

Este codigo Python, es el que se encarga del apartado back-end de nuestra implementación web en Flask. Lo primero que hace este codigo es la conexión a nuestra base de datos y a la colección especifica que vamos a utilizar, en nuestro caso la colección que contiene los datos a tiempo real de las distintas plazas. Nuestro codigo tiene una función que se encarga de dividir el bay-id para que podamos ordenarlos correctamente en nuestra web. La siguiente función es la que se encarga de renderizar el index.html y cargar los estilos que vayamos a utilizar. Por último tenemos la función que se encarga de hacer los llamados a la base de datos para conseguir los datos que vamos a mostrar en la interfaz web.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Aparcamientos</title>
<style>
    /* ======== ESTILOS GENERALES ======== */
    body {
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        margin: 0;
        background: linear-gradient(135deg, #e0eafc, #cfdef3);
        color: #333;
    }

    h1 {
        text-align: center;
        padding: 30px 10px;
        margin: 0;
        background: #2b5876;
        background: linear-gradient(90deg, #4e4376, #2b5876);
        color: white;
        font-weight: 600;
        letter-spacing: 1px;
        box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    }

    /* ======== CONTENEDOR ======== */
    .parking-container {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
        gap: 25px;
        padding: 30px;
        max-width: 1000px;
        margin: 0 auto;
    }

    /* ======== TARJETAS ======== */
    .parking-item {
        background-color: #fff;
        border-radius: 12px;
        padding: 20px;
        box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
        text-align: center;
        position: relative;
        cursor: pointer;
        transition: all 0.3s ease;
    }
```

Figura 72: Enter Caption

```
        .parking-item:hover {
            transform: translateY(-6px);
            box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);
        }

        .parking-item h3 {
            margin-top: 0;
            color: #2b5876;
        }

        .parking-item p {
            margin: 5px 0;
            font-size: 15px;
        }

        /* ======== COLORES DE ESTADO ======== */
        .parking-item.green {
            border-top: 5px solid #28a745;
        }

        .parking-item.red {
            border-top: 5px solid #dc3545;
        }

        /* ======== DETALLES OCULTOS ======== */
        .details {
            display: none;
            margin-top: 10px;
            background-color: #f8f9fa;
            padding: 10px;
            border-radius: 8px;
            font-size: 14px;
            text-align: left;
        }

        .parking-item:hover .details {
            display: block;
            animation: fadeIn 0.4s ease-in-out;
        }

        @keyframes fadeIn {
            from { opacity: 0; transform: translateY(-5px); }
            to { opacity: 1; transform: translateY(0); }
        }

        /* ======== FOOTER ======== */
        footer {
            text-align: center;
            padding: 15px;
            font-size: 14px;
            color: #555;
        }
    </style>
</head>
```

Figura 73: Enter Caption

```
</head>
<body>
    <h1>🚗 Estado de los Aparcamientos</h1>
    <div class="parking-container" id="parking-list">
        <!-- Los datos se actualizarán dinámicamente aquí -->
    </div>

    <footer>Actualización automática cada 10 segundos</footer>

    <script>
        // Función para actualizar los datos de los aparcamientos
        function updateParkingData() {
            fetch('/data')
                .then(response => response.json())
                .then(data => {
                    console.log('Datos recibidos:', data); // 👈 Depuración
                    const parkingList = document.getElementById('parking-list');
                    parkingList.innerHTML = '';

                    if (!data.length) {
                        parkingList.innerHTML = '<p style="text-align:center; color:#666;">No hay datos disponibles.</p>';
                        return;
                    }

                    data.forEach(item => {
                        const div = document.createElement('div');
                        div.className = 'parking-item ' + item.color;

                        div.innerHTML = `
                            <h3>Plaza Nº ${item.bay_id}</h3>
                            <h3>ID Parking:</strong> ${item.parking_id}<h3>
                            <p><strong>Ocupado:</strong> ${item.occupied ? 'Sí 🚗' : 'No ✅'}</p>
                            <div class="details">
                                <p><strong>Nivel:</strong> ${item.level}</p>
                                <p><strong>Último evento:</strong> ${item.last_event_ts}</p>
                                <p><strong>Temperatura:</strong> ${item.temperature_c} °C</p>
                                <p><strong>Batería:</strong> ${item.battery_pct} %</p>
                            </div>
                        `;

                        parkingList.appendChild(div);
                    });
                })
                .catch(error => console.error('Error al obtener los datos:', error));
        }

        // Actualización cada 10 segundos
        setInterval(updateParkingData, 10000);
        updateParkingData();
    </script>
</body>
</html>
```

Figura 74: Enter Caption

En nuestro index.html tendremos todos los estilos y como se verá nuestra pagina web. Además de una función que lee los datos que hemos cogido en el otro codigo para mostarlos.